

类增量学习 (CIL) 完整课程讲义

从零开始理解神经网络、灾难性遗忘、LwF、Replay、PODNet、MTD,
并读懂本项目的交接文档、实验代码、三线表和图表。

项目：CIFAR-100 类增量学习重启

范围：45 课 + 深度工作纸 + 公式表 + 项目地图 + 答辩题

版本：2026-07-17

使用说明

这不是一份只供浏览的论文摘要，而是一条从“完全不懂神经网络”走到“能独立解释项目、跑实验并在组会答辩”的学习路线。每一课都包含：学习目标、核心解释、与本项目的对应关系、练习和参考答案。第一次学习时按顺序读；第二次复习时，可直接跳到项目诊断、指标、图表和答辩部分。

本讲义严格区分三种东西：已经由项目日志证明的事实、来自论文与标准协议的知识、以及尚待正式实验验证的研究假设。当前 D0-D4 诊断实验已有真实结果；标准 B50-5S 的多方法三随机种子实验尚未完成，因此相关主表不会填入虚构数字。

最重要的学习习惯：每看到一个结论都追问三件事：它由哪份数据支持？与哪个对照组相比？是否可能由随机性、数据泄漏或实现错误造成？这三问比背方法名称更重要。

目录

使用说明	2
第一部分 从项目与数学开始	6
第 1 课 这个项目究竟在研究什么	7
第 2 课 够用的数学：向量、矩阵、概率和导数	9
第 3 课 Python：看懂训练脚本所需的最小知识	10
第 4 课 NumPy 与 PyTorch 张量	12
第二部分 神经网络从零入门	13
第 5 课 神经网络训练的完整流程	14
第 6 课 Softmax 与交叉熵 (CE)	15
第 7 课 卷积神经网络 (CNN)	16
第 8 课 VGG16 与 ResNet-32	17
第 9 课 CIFAR-100 数据集	18
第 10 课 训练集、验证集与测试集	19
第三部分 持续学习与类增量协议	20
第 11 课 持续学习、Task-IL、Domain-IL 与 Class-IL	21
第 12 课 灾难性遗忘	22
第 13 课 B50-5S 标准协议	23
第四部分 方法：从蒸馏到多教师	24
第 14 课 知识蒸馏 (KD)	25
第 15 课 KD 的共同平移盲区	26
第 16 课 LwF：不保存旧样本的学习	27
第 17 课 Replay 与 exemplar	28
第 18 课 Separated Softmax 与 SS-IL	29
第 19 课 PODNet：保护表示空间	30
第 20 课 MTD：真正的多教师蒸馏	31
第 21 课 历史 checkpoint 为什么不等于 MTD	32
第五部分 读懂本项目与类别替换故障	33
第 22 课 如何读交接文档	34
第 23 课 如何读旧 VGG 增量代码	35
第 24 课 bus -> train -> wolf 类别替换	36
第 25 课 受控实验与因果归因	37
第 26 课 D0-D4：怎样读真实诊断结果	38
第 27 课 诊断日志：不只记准确率	40
第六部分 指标、随机性与可信结论	42

第 28 课 overall、old、new 与逐类准确率	43
第 29 课 AA、AIA 与 forgetting	44
第 30 课 随机种子、均值与标准差	45
第七部分 工程：配置、日志与可复现性	46
第 31 课 YAML 配置怎么读	47
第 32 课 Checkpoint、环境与运行记录	48
第 33 课 raw -> tidy -> tables/figures	49
第八部分 三线表与图表解读	50
第 34 课 三线表：规则与研究问题	51
第 35 课 增量准确率曲线与遗忘曲线	52
第 36 课 稳定性-可塑性散点图	53
第 37 课 混淆矩阵	54
第 38 课 性能-成本图	56
第九部分 读论文与判断研究价值	57
第 39 课 怎样读 LwF 论文	58
第 40 课 怎样读 MTD 论文	59
第 41 课 怎样读蒸馏持续学习综述	60
第 42 课 创新边界与下一步决策	61
第十部分 组会叙事与独立答辩	62
第 43 课 把结果讲成一条可信故事	63
第 44 课 导师常问什么，怎样回答	64
第 45 课 最终实操考核：从零到可答辩	65
深度工作纸 把 45 课讲厚、讲透、讲到能答辩	66
工作纸 1：从一张图片到一个 logit	67
工作纸 2：CE 为什么会把新类推上去、把旧类压下去	68
工作纸 3：KD 的作用、盲区和共同平移推导	69
工作纸 4：CIL 协议、B50-5S 和指标手算	70
工作纸 5：方法矩阵一口气讲清	71
工作纸 6：旧 VGG 故障审计清单	72
工作纸 7：D0-D4 数字如何讲成证据链	73
工作纸 8：从 raw 日志到三线表	74
工作纸 9：八类图分别回答什么问题	75
工作纸 10：论文阅读到项目决策	76
工作纸 11：组会答辩 10 分钟脚本	77
工作纸 12：零基础学习路线和验收方式	78
附录 A 核心术语速查	79

附录 B 公式速查	80
附录 C 项目文件地图与运行顺序	81
附录 D 组会答辩题与回答骨架	82
附录 E 完成度清单	83
附录 F 资料来源与证据边界	84

第一部分 从项目与数学开始

这一部分先建立共同语言。你不需要提前学完高等数学；只要理解输入、参数、输出、误差和更新之间的关系，就能开始读神经网络代码。

第 1 课 这个项目究竟在研究什么

学完这一课，你应该会

- 用一句话解释类增量学习
- 说清输入、模型、输出和研究问题
- 区分旧故障复盘与正式标准实验

先看任务

普通图像分类假设所有类别一次性出现。类增量学习则把类别分批交给模型：模型先学一批旧类，之后只看到新类，还要继续识别所有学过的类。例如先学 50 类，再分 5 次各学 10 类，最终要在 100 类的同一个输出空间中做判断。

困难在于：学习新类需要改变参数，但改变参数可能破坏旧类知识。这叫稳定性与可塑性的冲突。稳定性是保住旧知识，可塑性是学会新知识。

本轮研究问题

第一条线是故障复盘：为什么旧 VGG 实验出现 bus 准确率从 0.57 变成 0，而新来的 train 迅速变高，下一阶段 train 又被 wolf 替换。第二条线是标准复现：在 CIFAR-100、ResNet-32、B50-5S 协议上比较单教师、多教师与分类偏差校正方法。

本轮目标不是急着发明新算法，而是先得到可信、可复现、能解释的现象。只有基线可靠，后续创新才有意义。

零基础加厚讲解

先修回顾：你只需要先抓住一个主语：模型。模型接收图片，输出每个类别的分数。类增量学习研究的是：新类别一批批来时，模型怎样既学新类，又不把旧类判错。

直觉解释

把这个项目想成一个长期考试。普通分类像期末考试前把所有题型一次性给你；类增量学习像老师今天只教 50 类，过几天再教 10 类，但最终考试要求 100 类混在一起判断。难点不是“会不会学 train”，而是“学 train 时会不会把 bus 忘掉”。

因此本项目有两条线。第一条是故障诊断线：旧 VGG 实验为什么出现 bus -> train -> wolf 的替换。第二条是标准复现线：在 CIFAR-100 B50-5S 协议下，用统一日志和表图比较 LwF、Replay、PODNet、MTD、SS-IL 等方法。

放到项目里怎么用

- 读交接文档时，不要先记方法名。先画一张四格图：旧实现发生了什么、证据数字是什么、怀疑原因是什么、还需要哪个对照实验。
- 正式汇报时，旧 VGG 数字只证明“旧流程有故障”，不能放进标准主结果表与 PODNet/MTD 排名。

常见误区

- 误区：导师说旧结果不好，就把旧结果删掉。正确做法是把旧结果转成故障案例，用它说明为什么要重启。
- 误区：一上来追求创新。当前更重要的是先建立可信协议，否则任何新方法数字都站不住。

分层练习：闭卷写出本项目一句话目标：在类增量学习中解释旧类被新类替换的机制，并在标准 CIFAR-100 协议上建立可信基线。

放回本项目：旧目录 VGG16_CIL 只作为故障案例；新项目 cil_restart 承担标准协议、统一日志和自动表图。

练习与自检：如果导师问“你们到底解决什么问题”，请用两句话回答。

参考答案：我们研究模型连续学习新类别时如何避免旧类被新类替换。先用 D0-D4 定位旧实现的分类偏差和损失缺陷，再在标准 CIFAR-100 CIL 协议上比较可靠基线。

第 2 课 够用的数学：向量、矩阵、概率和导数

学完这一课，你应该会

- 读懂 x 、 W 、 b 、 z 的含义
- 理解概率归一化
- 知道梯度告诉参数往哪里改

向量与矩阵

一张彩色图片可看成很多数字组成的三维数组。经过网络后，它被压缩成一个特征向量 x 。分类器通常计算 $z = Wx + b$ ： W 的每一行对应一个类别， b 是每类偏置， z 是 logits，也就是尚未归一化的类别分数。

矩阵乘法的直觉是“加权组合”。某个类别的权重行与特征方向越相似，该类别 logit 越大。

概率与导数

Softmax 把任意 logits 变成和为 1 的概率。导数描述输入稍微变化时输出变化多快，梯度则收集损失对所有参数的导数。优化器沿着负梯度方向更新参数，使损失下降。

你不必一开始手算大型矩阵。需要掌握的是因果链：参数改变 $\rightarrow$ logits 改变 $\rightarrow$ 概率改变 $\rightarrow$ 损失改变；反向传播把这条链倒着求导。

零基础加厚讲解

先修回顾：这一课只需要四个数学对象：图片特征 x 、分类器权重 W 、偏置 b 、输出分数 z 。后面所有 CE、KD、logit gap 都围绕它们转。

直觉解释

$z = Wx + b$ 可以理解为“每个类别拿自己的模板去匹配图片特征”。 W 的第 c 行越像当前图片特征，第 c 类分数越高。softmax 只是把这些分数换成相对概率，但模型最后常常直接看哪个 logit 最大。

概率不是绝对分数，而是相对关系。bus logit 本身不低也可能输给 train，只要 train logit 更高。类别替换的关键正是新类 logit 整体抬高、旧类 logit 整体下移，导致决策边界被推过去。

放到项目里怎么用

- 看诊断图时，把 old/new logit gap 当作温度计： $gap = \text{mean}(\text{new logits}) - \text{mean}(\text{old logits})$ 。gap 越大，模型越偏向新类。
- 分类器权重范数和偏置不是装饰指标。若新类行权重或偏置异常变大，它可能直接造成新类吸走旧类样本。

常见误区

- 误区：准确率下降一定是特征忘了。也可能是最后分类器尺度偏了，特征还在但旧类分数被压低。
- 误区：softmax 概率低说明模型完全不会。要同时看 logits、混淆矩阵和 per-class accuracy。

分层练习：给定 bus logit=2、train logit=5，模型会判 train。即使 bus 的绝对分数为正，只要 train 更高，bus 就会被替换。

放回本项目：项目中的 old/new logit gap、分类器权重范数和 CE/KD 梯度，分别对应 z 、 W 和损失对参数/输出的导数。

练习与自检：为什么分类器 W 的第 c 行可以理解为类别 c 的“模板”？

参考答案：因为类别 c 的 logit 是 W_c 与特征 x 的内积再加偏置；两者方向越匹配，分数通常越高。

第 3 课 Python：看懂训练脚本所需的最小知识

学完这一课，你应该会

- 看懂变量、列表、字典、循环与函数
- 理解类和对象的基本用法
- 知道配置如何传给训练程序

最常见结构

变量给数据起名字；列表保存有顺序的对象；字典用键查值；for 循环重复处理 batch 或 epoch；函数把一段逻辑封装起来；class 定义模型、数据集或训练器。训练脚本常见结构是：读取配置 -> 构建数据 -> 构建模型 -> 进入 epoch/batch 循环 -> 计算损失 -> 保存日志。

看到 import 时先判断模块职责，不必立刻读完全部依赖。阅读顺序应从程序入口和配置开始，再追踪 model、loss、optimizer 和 dataloader。

调试原则

先打印形状和关键数值，再猜原因。对张量重点看 shape、dtype、device、min/max、是否包含 NaN。对配置重点看数据路径、类别顺序、阶段数、随机种子和 checkpoint。

零基础加厚讲解

先修回顾：你不需要成为 Python 工程师才能接项目，但必须能看懂训练脚本的主干：配置、数据、模型、损失、优化器、日志。

直觉解释

训练脚本通常是一条流水线：读取配置 -> 构造数据集 -> 构造模型 -> 进入 epoch 循环 -> 每个 batch 计算 loss -> backward -> optimizer.step -> 保存日志。读代码时先找这条主线，再看细节。

字典和配置文件很关键。method、seed、scenario、task_id、memory_size 这些字段决定一次实验到底是什么实验。若字段缺失，后面三线表就无法判断哪些结果能放在一起比较。

放到项目里怎么用

- 旧脚本中要特别追踪：分类器扩展代码、teacher/student 前向、KD loss、optimizer 包含哪些参数、是否用测试准确率选 checkpoint。
- 新工作区的目标是让每次运行都留下 raw JSON。你以后看到一张表，要能反查到它来自哪个 seed 的哪个 raw 文件。

常见误区

- 误区：从第一行 import 开始逐行啃。更有效的是先找 main/train 函数和训练循环。
- 误区：只看 loss 数值。loss 降低可能只是新类学好了，旧类已经被压没了。

分层练习：在任何训练脚本里先找五个词：dataset、model、criterion/loss、optimizer、logger。找不到其中一个，就先别谈结果。

放回本项目：旧脚本 train_CIL12.py 的关键不是每行语法，而是找出：分类器如何扩展、哪些参数参与优化、训练数据含哪些类、CE/KD 如何组合。

练习与自检：for images, labels in loader: 这一行在做什么？

参考答案：它从数据加载器逐批取出图像张量和对应标签，循环体对每个 batch 完成前向、损失、反向与更新。

第 4 课 NumPy 与 PyTorch 张量

学完这一课，你应该会

- 分清数组与张量
- 读懂 NCHW 形状
- 理解 device、detach 和 no_grad

张量是什么

张量就是多维数字容器。图像 batch 常用 NCHW：N 是样本数，C 是通道数，H/W 是高宽。CIFAR-100 原图为 32×32 彩色图，单张通常是 $3 \times 32 \times 32$ 。RGB/BGR 错误就是三个通道的语义顺序弄反。

PyTorch 张量除保存数值，还能记录计算图并自动求导。requires_grad=True 表示需要跟踪梯度；loss.backward() 会把梯度写入相关参数的 .grad。

三个常见陷阱

device 不一致会报错，例如模型在 GPU、数据在 CPU。detach() 会切断梯度路径，教师输出通常应 detach；但若把学生相关项错误 detach，就可能导致损失对学生没有梯度。torch.no_grad() 适合评估和教师前向，可降低显存。

零基础加厚讲解

先修回顾：张量就是多维数字表。CIFAR-100 彩色图通常是 $3 \times 32 \times 32$ ，batch 后变成 $N \times 3 \times 32 \times 32$ 。

直觉解释

RGB/BGR 错误不是小格式问题。红绿蓝通道顺序错了，模型看到的颜色语义就变了；如果同时换优化器、加 epoch、修通道，准确率变化就无法归因。

requires_grad、detach、no_grad 决定梯度能不能流。教师模型输出通常要 detach，因为不训练教师；学生输出不能 detach，否则 KD loss 对学生没有作用。

放到项目里怎么用

- 交接问题中有一项是双教师损失存在无学生梯度项。排查办法不是看 loss 是否有数值，而是检查 loss.backward() 后学生参数或 logits 的 grad 是否非零。
- 诊断脚本记录 CE/KD 对分类器各输出行的梯度，就是为了证明哪些类别行被推高、哪些被压低。

常见误区

- 误区：loss 打印出来就说明训练有效。常数 loss 也能打印，但不会更新学生。
- 误区：detach 越多越省显存越好。detach 错对象会直接切断学习信号。

分层练习：一句话判断：教师输出可以 no_grad，学生输出必须保留计算图，最终 loss 必须能回传到学生参数。

放回本项目：双教师旧代码中“无学生梯度的损失项”就是计算图断开后，数值看似存在，却无法推动学生参数更新。

练习与自检：如果一个损失项数值为 2.0，但它对所有学生参数的梯度都是 0，它能训练学生吗？

参考答案：不能。优化器只依据梯度更新；有数值但无梯度，相当于把常数加到总损失上。

第二部分 神经网络从零入门

下面把一次训练拆开讲。只要你能画出“数据 -> 网络 -> logits -> 损失 -> 梯度 -> 更新”的闭环，就已具备理解本项目的骨架。

第 5 课 神经网络训练的完整流程

学完这一课，你应该会

- 说出前向传播和反向传播
- 理解 epoch、batch 和 optimizer
- 区分训练模式与评估模式

一次 batch

第一步把图像送入模型得到 logits；第二步用标签计算损失；第三步 `optimizer.zero_grad()` 清空旧梯度；第四步 `loss.backward()` 反向传播；第五步 `optimizer.step()` 更新参数。这个过程对所有 batch 重复一次称为一个 epoch。

训练时 `model.train()` 会启用训练行为，例如 BatchNorm 更新统计量、Dropout 随机丢弃；评估时 `model.eval()` 固定这些行为。忘记切换会让结果不稳定。

学习率与优化器

学习率控制每次更新步长。太大可能震荡，太小训练缓慢。SGD 常用于图像分类；Adam 自适应调整步长。若修通道、换优化器、加 epoch 同时发生，即使准确率变好，也无法知道是哪项改变起作用。

零基础加厚讲解

先修回顾：一次训练更新由 forward、loss、zero_grad、backward、step 组成。epoch 是把训练集完整走一遍。

直觉解释

神经网络训练不是直接告诉模型“记住 bus”，而是通过损失函数给参数一个方向。CE 告诉模型提高正确类分数、压低其他类分数；KD 告诉学生尽量像教师。

如果新阶段训练数据只有 train，CE 每个 batch 都在强调 train 是正确类。旧类没有样本参与 CE，就没有同等力量把 bus 拉回来。

放到项目里怎么用

- D0-D4 的设计核心是单变量诊断：D0 当前实现，D1 只修权重继承，D2 冻结旧分类器，D3 加 replay，D4 加 separated softmax。
- 你要能解释为什么诊断只跑前 6 epoch：目的是观察替换发生过程，而不是追求最终最高准确率。

常见误区

- 误区：epoch 越多越可靠。若训练信号偏了，更多 epoch 只会更稳定地走向错误。
- 误区：固定 seed 没意义。诊断实验固定 seed 是为了让不同组之间只差一个因素。

分层练习：写出一次 batch 的顺序：前向得到 logits，计算 CE/KD，清梯度，反向传播，优化器更新。

放回本项目：交接文档中的第一次大问题正是多项改动同时发生，破坏了单变量归因。D0-D4 因此要求固定种子并逐项改变。

练习与自检：请按正确顺序排列：step、backward、forward、zero_grad、loss。

参考答案：forward -> loss -> zero_grad -> backward -> step。

第 6 课 Softmax 与交叉熵 (CE)

学完这一课，你应该会

- 从 logits 得到概率
- 理解 CE 为什么抬高正确类
- 解释新类单独训练造成的偏差

Softmax

对第 c 类, $p_c = \exp(z_c) / \sum_j \exp(z_j)$ 。所有类别共同竞争：提高一个 logit 会相对降低其他类别概率。Softmax 对所有 logits 同时加同一个常数不变，这是数值稳定实现常先减去最大值的原因。

交叉熵梯度

单样本标签为 y 时, $CE = -\log p_y$ 。它对 logit z_j 的梯度是 $p_j - 1[j=y]$ 。因此正确类梯度为负，梯度下降会把正确类 logit 提高；其余类梯度为正，会被压低。

如果训练 batch 只有新类，旧类永远是“其余类”。CE 会反复压低所有旧类 logits，同时抬高新类 logit。这正是类别替换的重要动力。

零基础加厚讲解

先修回顾： logits 是原始分数，softmax 把它们变成概率，CE 用真实标签惩罚正确类概率太低。

直觉解释

CE 的梯度公式很重要：对 logit z_j ，梯度约等于 $p_j - 1[j=y]$ 。真实类 y 的梯度为 $p_y - 1$ ，通常是负数；梯度下降会提高它。其他类梯度为 p_j ，梯度下降会降低它们。

这解释了新类单独训练的危险：当 batch 全是 train，train 是正确类，bus、旧类都属于“其他类”，会被 CE 系统性压低。

放到项目里怎么用

- bus $\rightarrow$ train 替换不是神秘现象。若 train logit 被 CE 一直抬高，而 bus 没有 replay 样本或有效约束，bus 样本最终也会被判成 train。
- D3 加每类 20 个旧样本 replay 后 old accuracy 从接近 0 恢复到约 58.73，说明“训练数据只有新类”是强因素。

常见误区

- 误区：CE 只提高正确类，不影响其他类。softmax 竞争结构使它同时压低其他类。
- 误区：只要 KD 存在就能保护旧类。KD 的定义范围和梯度路径决定它能保护什么。

分层练习：若当前样本标签是 train，bus 不是真实类，则 CE 对 bus logit 的梯度方向会让 bus logit 下降。

放回本项目：11- $\rightarrow$ 12 类阶段只有 train 新类数据时，CE 将 train 当正确类，并把 bus 等全部旧类当负类；因此只靠旧类内部 KD 不足以保持跨组决策尺度。

练习与自检：标签是新类 train 时，旧类 bus 的 CE logit 梯度通常是正还是负？参数更新后 bus logit 倾向上升还是降？

参考答案：梯度通常为负；梯度下降会沿负梯度更新，所以 bus logit 倾向降低。

第 7 课 卷积神经网络 (CNN)

学完这一课，你应该会

- 理解卷积核与特征图
- 知道池化和感受野的作用
- 区分特征提取器与分类器

卷积的直觉

卷积核是一个小窗口，在图像上滑动并寻找局部模式，例如边缘、纹理。浅层学简单模式，深层把它们组合成部件和类别相关语义。卷积共享参数，因此比给每个像素单独连一层更适合图像。

网络的两部分

特征提取器把图像变成向量 x ；最后分类器用 $Wx + b$ 输出各类 logits。增量学习中两部分都会漂移：特征变化会让旧类样本的位置改变，分类器权重变化会改变决策边界。只观察最终准确率无法分清是哪部分出了问题。

零基础加厚讲解

先修回顾：反向传播就是把总损失对每个参数的影响算出来，优化器根据这些梯度改参数。

直觉解释

梯度不是抽象数学符号，在项目里就是 `.grad`。你可以检查分类器每一行的梯度范数，看新类行、旧类行分别被怎样推动。

如果一个损失项没有连到学生输出，它的数值再大也只是加了一个常数。优化器只看梯度，不看你主观认为这个损失重要。

放到项目里怎么用

- 双教师代码的问题之一就是某些损失项缺少学生梯度。验证方法：对单个 batch 分别 backward CE 和 KD，记录每个输出行 grad norm。
- 图 3 的 CE/KD 梯度方向图应该回答：CE 是否主要推高新类并压低旧类，KD 是否真的约束了学生旧类输出。

常见误区

- 误区：总 loss 包含某项，就说明该项生效。必须检查梯度。
- 误区：梯度越大越好。过大的梯度可能造成新类过度支配或训练不稳定。

分层练习：诊断一个 KD loss 是否有效：看 student logits 是否参与 loss，看 backward 后 student 参数 grad 是否非零。

放回本项目：D0-D4 记录分类器行权重范数和 logits；PODNet 进一步约束中间特征，目的就是同时保护表示空间。

练习与自检：为什么冻结分类器不一定完全解决遗忘？

参考答案：因为特征提取器仍可能改变；旧图像得到的新特征不再与旧分类器权重匹配。

第 8 课 VGG16 与 ResNet-32

学完这一课，你应该会

- 知道两种网络的结构差别
- 理解残差连接
- 说明为什么正式实验换用 ResNet-32

VGG

VGG 通过连续 3×3 卷积和池化逐步加深，结构直观但参数和计算量较大。旧项目把 CIFAR 图像转 JPEG 并放大到 224×224 ，再用 VGG16；这引入了额外的数据变换和成本。

ResNet

ResNet 的残差块学习 $F(x)$ ，输出 $x + F(x)$ 。捷径连接让梯度更容易传播，深层训练更稳定。CIFAR 版 ResNet-32 直接处理 32×32 图像，是增量学习论文中常见的标准骨干之一。

更换骨干会改变绝对准确率，因此 VGG 旧结果不能与 ResNet 标准结果混在同一排名表。

零基础加厚讲解

先修回顾：卷积网络把图片从像素变成特征，再由分类器输出类别分数。VGG 和 ResNet 都属于这个流程。

直觉解释

卷积层像一组可学习的滤镜，早期检测边缘、颜色、纹理，后期组合成更抽象的物体线索。分类器最后把这些线索映射到类别。

VGG16 参数多、输入常用 224×224 ；CIFAR-100 原图是 32×32 。本轮标准复现使用 ResNet-32 和原始 CIFAR-100，不再把图片转 JPEG 或放大到 224。

放到项目里怎么用

- 旧 VGG 实验只用于故障复盘；正式 B50-5S 不能混入 VGG 数字，否则模型结构和数据处理协议都不一致。
- ResNet-32 是持续学习论文中常见 CIFAR backbone，便于和官方 CLearning 配置对齐。

常见误区

- 误区：更大的网络一定更好。协议不一致时，网络大小会成为混杂因素。
- 误区：把 CIFAR 放大成 224 只是增强。它改变了输入分布、计算量和可比性。

分层练习：汇报时说清楚：旧线是 VGG 故障案例，正式线是 CIFAR-100 原图 + ResNet-32 标准协议。

放回本项目：正式协议固定 CIFAR-100 原图和 ResNet-32；旧 VGG 实验保留为故障案例，不进入主结果表。

练习与自检：为什么不应把旧 VGG 数字和新 ResNet 数字直接排名？

参考答案：因为骨干、输入分辨率、数据处理和训练协议不同，差异无法归因于增量学习方法本身。

第 9 课 CIFAR-100 数据集

学完这一课，你应该会

- 理解 100 类与数据划分
- 知道训练增强和标准化
- 识别 RGB/BGR 与 JPEG 风险

数据结构

CIFAR-100 含 100 个细粒度类别，每张图 32×32 、RGB 三通道。常见官方划分为每类 500 张训练图和 100 张测试图。类增量协议还会规定类别出现顺序。

数据处理

训练时可使用随机裁剪、水平翻转和标准化；评估时通常只标准化。所有方法必须共享同一处理流程。把原图转 JPEG 会引入压缩变化，放大到 224 也改变计算量。OpenCV 默认 BGR，而 PIL/torchvision 常按 RGB，混用时必须显式转换。

零基础加厚讲解

先修回顾：分类器最后一层通常是一个线性层。增加新类时，输出维度要从旧类数扩到旧类数+新类数。

直觉解释

扩展分类器时，旧类行应该继承旧模型权重，新类行单独初始化。若整个输出层重新初始化，旧类模板直接丢失，旧类准确率归零就不奇怪。

即使正确复制旧权重，如果预热或训练阶段更新整个分类层，新类数据的 CE 仍可能推动旧类行一起漂移。

放到项目里怎么用

- 交接文档指出 10->11 类时整个输出层被重新初始化；11->12、12->13 虽复制旧权重，但预热阶段更新了整个分类层。
- D1 分离初始化错误影响；D2 冻结旧分类器，只训练新类输出行，用来验证分类层漂移是否关键。

常见误区

- 误区：复制权重后就安全。还要看 optimizer 是否包含旧分类器行。
- 误区：只冻结 backbone。类别替换常发生在最后分类器尺度和偏置上。

分层练习：扩 11 到 12 类的正确基本动作：新建 12 行，复制前 11 行，初始化第 12 行，并明确冻结/更新策略。

放回本项目：新项目直接使用 torchvision 原始 CIFAR-100，避免重复 JPEG 转换与 RGB/BGR 混乱。

练习与自检：如果训练和测试都错误地使用 BGR，结果一定完全不能用吗？

参考答案：不一定完全失败，模型可能适应一致的错误顺序；但若预训练、均值方差或不同阶段的通道处理不一致，就会严重破坏可比性。最重要的是明确并固定协议。

第 10 课 训练集、验证集与测试集

学完这一课，你应该会

- 区分三个数据集合
- 理解测试泄漏
- 知道固定 epoch 与验证选择的区别

职责分离

训练集用于计算梯度；验证集用于调超参数、早停或选择 checkpoint；测试集只在方案确定后评价泛化。若每个 epoch 都看测试准确率并选择最高者，测试集事实上参与了训练决策。

本项目规则

标准复现优先使用论文/官方配置的固定 epoch 和学习率计划。若确需选 checkpoint，应从训练数据中划分验证集，并对所有方法使用相同规则。最终测试一次并完整报告。

零基础加厚讲解

先修回顾：评估就是在测试集上算预测是否正确。CIL 评估必须在所有已学类别中选最大分数。

直觉解释

overall accuracy 告诉你总正确率，但掩盖旧类和新类差异。old accuracy 低、new accuracy 高，说明模型偏新；二者都低，可能表示特征或训练整体失败。

per-class accuracy 和混淆矩阵能显示具体哪类被吸走。bus 准确率 0 还不够，要看 bus 被预测成 train 还是分散到多类。

放到项目里怎么用

- D0-D4 每个 epoch 记录 bus、train、旧类平均准确率，以及 bus 被错误预测成哪个类别，就是为了避免只看 overall。
- 最终图 7 混淆矩阵要按增量阶段分组，标出旧类被预测为最新类别的区域。

常见误区

- 误区：最终 accuracy 高就说明方法好。可能它只学会了新类，旧类已经忘掉。
- 误区：测试集可以用来挑最佳 epoch。这样会测试泄漏，正式结论不可信。

分层练习：当 old=0、new=100、overall 看似不低时，你要指出这是严重新类偏置，不是成功学习。

放回本项目：旧实现使用测试准确率选择最佳 epoch，造成测试泄漏；代码与协议审计表必须标记该问题。

练习与自检：为什么“没有对测试集反向传播”仍可能发生测试泄漏？

参考答案：只要测试结果影响了模型、超参数或 checkpoint 的选择，信息就已泄漏，不需要直接反向传播。

第三部分 持续学习与类增量协议

第 11 课 持续学习、Task-IL、Domain-IL 与 Class-IL

学完这一课，你应该会

- 区分三种场景
- 理解单头分类
- 知道任务 ID 是否可用

三个场景

Task-IL 在测试时知道样本属于哪个任务，可选择对应输出头；Domain-IL 的类别不变，但输入分布变化；Class-IL 持续增加新类别，测试时不给任务 ID，模型必须在所有已学类别中单头竞争。

为什么 Class-IL 更难

没有任务 ID 就不能先缩小候选范围。旧类和新类 logits 必须处于可比较的尺度；因此训练数据不平衡和分类器偏置会直接造成“所有样本都预测成最新类”。

零基础加厚讲解

先修回顾：从这里开始，重点从普通神经网络转到持续学习。你要不断追问：训练时看到了哪些类别，测试时要求分哪些类别。

直觉解释

持续学习不是单个算法，而是一组协议。Task-IL、Domain-IL、Class-IL 的难度不同；本项目采用单头 Class-IL，测试时不给任务 ID。

灾难性遗忘有多种来源：表示被改坏、旧数据缺失、分类器偏向新类。一个实验如果不能分开这些因素，就很难得出可靠结论。

放到项目里怎么用

- B50-5S 的含义是首任务 50 类，后续 5 个阶段每阶段 10 类，总共 6 个评价点。
- 每类 20 个 exemplar 是允许边界，意味着 Replay/SS-IL/PODNet 可以用小记忆库，但不能偷看完整旧训练集。

常见误区

- 误区：把 Task-IL 结果拿来和 Class-IL 比。Task-IL 给任务 ID，难度低很多。
- 误区：只报告平均值不报告阶段曲线。持续学习关心过程，不能只看终点。

分层练习：画出六个阶段：50、60、70、80、90、100 类，并标出每阶段测试要在已学所有类中分类。

本课专属任务：本课专属任务：把 Task-IL、Domain-IL、Class-IL 写成三列表，并标出本项目为什么属于单头 Class-IL。

放回本项目：本项目是单头 Class-IL。若把不同阶段分别用不同输出头测试，会把问题变简单，不能与标准 CIL 结果比较。

练习与自检：测试时给出任务 ID 并只在该任务类别中选最大值，属于哪种设定？

参考答案：通常属于 Task-IL，而不是本项目要求的单头 Class-IL。

第 12 课 灾难性遗忘

学完这一课，你应该会

- 解释遗忘的三个来源
- 区分特征漂移和分类偏差
- 知道为什么新类准确率高不代表成功

三个来源

第一，参数共享导致新任务梯度覆盖旧知识；第二，没有旧数据时，模型不知道旧分布；第三，类别不平衡使分类器偏向新类。前两者偏向表示遗忘，第三者是决策偏差，它们可能同时发生。

如何观察

整体准确率可能掩盖问题，应分开报告 old accuracy 和 new accuracy；还要看逐类准确率、混淆矩阵、old/new logit gap 和历史最佳到当前的下降。若新类接近 100%、旧类接近 0%，这不是学得好，而是发生极端偏置。

零基础加厚讲解

先修回顾：从这里开始，重点从普通神经网络转到持续学习。你要不断追问：训练时看到了哪些类别，测试时要求分哪些类别。

直觉解释

持续学习不是单个算法，而是一组协议。Task-IL、Domain-IL、Class-IL 的难度不同；本项目采用单头 Class-IL，测试时不给任务 ID。

灾难性遗忘有多种来源：表示被改坏、旧数据缺失、分类器偏向新类。一个实验如果不能分开这些因素，就很难得出可靠结论。

放到项目里怎么用

- B50-5S 的含义是首任务 50 类，后续 5 个阶段每阶段 10 类，总共 6 个评价点。
- 每类 20 个 exemplar 是允许边界，意味着 Replay/SS-IL/PODNet 可以用小记忆库，但不能偷看完整旧训练集。

常见误区

- 误区：把 Task-IL 结果拿来和 Class-IL 比。Task-IL 给任务 ID，难度低很多。
- 误区：只报告平均值不报告阶段曲线。持续学习关心过程，不能只看终点。

分层练习：画出六个阶段：50、60、70、80、90、100 类，并标出每阶段测试要在已学所有类中分类。

本课专属任务：本课专属任务：用 bus/train/wolf 例子解释灾难性遗忘和分类偏差的区别：前者是旧知识损坏，后者是决策尺度偏向新类。

放回本项目：bus->train->wolf 体现的不是普通小幅遗忘，而是最新类别在单头决策中替换上一个类别。

练习与自检：模型最终 overall=50%，能否判断它是否平衡？

参考答案：不能。可能旧类 0%、新类 100%，也可能两者都 50%。必须拆分 old/new 和逐类结果。

第 13 课 B50-5S 标准协议

学完这一课，你应该会

- 读懂 B50-5S
- 计算每阶段类别数
- 理解 exemplar memory

协议含义

B50-5S 表示 base task 先学 50 类，之后有 5 个增量阶段，每阶段加入 10 类：50、60、70、80、90、100。每一阶段测试当前已学的全部类别。类别顺序和随机种子必须记录。

记忆库

每类 20 个 exemplar 表示为每个已学类别保存 20 张代表样本。总记忆随类别数增长时，最终为 2000 张；如果论文采用固定总容量，则每类配额会下降。两种协议不可混淆。

零基础加厚讲解

先修回顾：从这里开始，重点从普通神经网络转到持续学习。你要不断追问：训练时看到了哪些类别，测试时要求分哪些类别。

直觉解释

持续学习不是单个算法，而是一组协议。Task-IL、Domain-IL、Class-IL 的难度不同；本项目采用单头 Class-IL，测试时不给任务 ID。

灾难性遗忘有多种来源：表示被改坏、旧数据缺失、分类器偏向新类。一个实验如果不能分开这些因素，就很难得出可靠结论。

放到项目里怎么用

- B50-5S 的含义是首任务 50 类，后续 5 个阶段每阶段 10 类，总共 6 个评价点。
- 每类 20 个 exemplar 是允许边界，意味着 Replay/SS-IL/PODNet 可以用小记忆库，但不能偷看完整旧训练集。

常见误区

- 误区：把 Task-IL 结果拿来和 Class-IL 比。Task-IL 给任务 ID，难度低很多。
- 误区：只报告平均值不报告阶段曲线。持续学习关心过程，不能只看终点。

分层练习： 画出六个阶段：50、60、70、80、90、100 类，并标出每阶段测试要在已学所有类中分类。
本课专属任务： 本课专属任务：画出 B50-5S 的六个测试点，并写出每个测试点包含多少个已学类别。
放回本项目： 正式配置固定 ResNet-32、batch size 128、memory batch 32、种子 1993/1994/1995，并以官方 CLearning 固定提交为准。
练习与自检： 到 Task 3（从 Task 0 计数）一共学了多少类？
参考答案： Task 0 为 50 类，Task 1/2/3 各加 10 类，所以 Task 3 共 80 类。

第四部分 方法：从蒸馏到多教师

第 14 课 知识蒸馏 (KD)

学完这一课，你应该会

- 理解教师与学生
- 读懂温度和 KL
- 知道 KD 保留什么信息

基本思想

教师是更新前的旧模型，学生是要学习新类的当前模型。对同一输入，教师给旧类产生软分布，学生被要求接近该分布。常见损失为 $KL(p_{teacher}^T || p_{student}^T)$ ，T 是温度；较高温度使分布更平滑，暴露类别相似关系。

局限

教师只会自己学过的类；它不能提供从未见过的新类知识。若蒸馏输入只有新类图像，教师输出也可能缺少覆盖旧分布的信息。KD 主要保持相对关系，并不天然解决新旧类整体尺度。

零基础加厚讲解

先修回顾：从这里开始，重点从普通神经网络转到持续学习。你要不断追问：训练时看到了哪些类别，测试时要求分哪些类别。

直觉解释

持续学习不是单个算法，而是一组协议。Task-IL、Domain-IL、Class-IL 的难度不同；本项目采用单头 Class-IL，测试时不给任务 ID。

灾难性遗忘有多种来源：表示被改坏、旧数据缺失、分类器偏向新类。一个实验如果不能分开这些因素，就很难得出可靠结论。

放到项目里怎么用

- B50-5S 的含义是首任务 50 类，后续 5 个阶段每阶段 10 类，总共 6 个评价点。
- 每类 20 个 exemplar 是允许边界，意味着 Replay/SS-IL/PODNet 可以用小记忆库，但不能偷看完整旧训练集。

常见误区

- 误区：把 Task-IL 结果拿来和 Class-IL 比。Task-IL 给任务 ID，难度低很多。
- 误区：只报告平均值不报告阶段曲线。持续学习关心过程，不能只看终点。

分层练习：画出六个阶段：50、60、70、80、90、100 类，并标出每阶段测试要在已学所有类中分类。

本课专属任务：本课专属任务：解释每类 20 exemplar 为什么既能提供旧类信号，又不能等同于重新拿到完整旧数据。

放回本项目：10 类教师没有 bus 知识，因此无法用它保护 bus；历史 checkpoint 的类别覆盖范围必须审计。

练习与自检：为什么教师不能教会学生一个教师从未训练过的类别？

参考答案：因为教师对该类别没有对应输出和监督形成的知识；其输出无法提供可信目标。

第 15 课 KD 的共同平移盲区

学完这一课，你应该会

- 证明旧类 softmax 对共同平移不变
- 连接到 logit gap
- 解释 KD 为何阻止不了整体下移

数学证明

若只对旧类做 softmax，并给所有旧类 logits 加常数 a ，则 $\exp(z_i+a)/\sum_j \exp(z_j+a) = \exp(a)\exp(z_i)/(\exp(a)\sum_j \exp(z_j))$ ， $\exp(a)$ 约掉，分布完全不变。KL 蒸馏因此几乎不变。

决策却会改变

单头预测比较旧类和新类的原始 logits。如果所有旧类共同下降 10，而旧类内部相对顺序不变，旧类 KD 看不出差异，但新类可能超过所有旧类。 $\text{old/new logit gap} = \text{mean}(\text{new logits}) - \text{mean}(\text{old logits})$ 正是测量这种尺度漂移。

零基础加厚讲解

先修回顾：从这里开始，重点从普通神经网络转到持续学习。你要不断追问：训练时看到了哪些类别，测试时要求分哪些类别。

直觉解释

持续学习不是单个算法，而是一组协议。Task-IL、Domain-IL、Class-IL 的难度不同；本项目采用单头 Class-IL，测试时不给任务 ID。

灾难性遗忘有多种来源：表示被改坏、旧数据缺失、分类器偏向新类。一个实验如果不能分开这些因素，就很难得出可靠结论。

放到项目里怎么用

- B50-5S 的含义是首任务 50 类，后续 5 个阶段每阶段 10 类，总共 6 个评价点。
- 每类 20 个 exemplar 是允许边界，意味着 Replay/SS-IL/PODNet 可以用小记忆库，但不能偷看完整旧训练集。

常见误区

- 误区：把 Task-IL 结果拿来和 Class-IL 比。Task-IL 给任务 ID，难度低很多。
- 误区：只报告平均值不报告阶段曲线。持续学习关心过程，不能只看终点。

分层练习：画出六个阶段：50、60、70、80、90、100 类，并标出每阶段测试要在已学所有类中分类。

本课专属任务：本课专属任务：给 FineTune、LwF、Replay 各写一句 sanity 作用，说明它们为什么不是最终主结论。

放回本项目：诊断日志数值验证共同平移后 KD 变化约 1.19×10^{-7} ，可视为浮点误差范围内不变。

练习与自检：教师旧类 logits 为 [3,2,1]，学生为 [-7,-8,-9]。旧类 softmax 是否相同？单头决策风险是什么？

参考答案：二者只相差共同常数 -10，旧类 softmax 相同；但学生旧类整体过低，任何新类较高 logit 都可能把旧样本抢走。

第 16 课 LwF：不保存旧样本的学习

学完这一课，你应该会

- 说明 LwF 的训练信号
- 知道其优点与局限
- 避免混淆多任务与单头协议

方法

Learning without Forgetting 在新任务数据上，同时用真实新标签训练新任务，并让学生在旧输出上模仿旧模型。它不要求保存旧训练样本，适合有隐私或存储限制的场景。

在 CIL 中的局限

新类图像不等于旧类分布；教师只在新类图像上提供旧输出，可能不足以保持旧表示。更关键的是，单头 CIL 中新旧类共同竞争，LwF 的原始多头设定与评估方式必须仔细适配。

零基础加厚讲解

先修回顾：这一部分学习方法。不要先背名字，先问每个方法用什么信号保护旧知识：输出、特征、样本、教师，还是分类偏差校正。

直觉解释

LwF 用旧模型输出约束学生；Replay 把旧样本混回训练；SS-IL 用 separated softmax 缓解新旧类竞争偏差；PODNet 蒸馏中间特征；MTD 构造多个教师增加蒸馏信息。

方法有效必须同时满足两个条件：训练信号确实能约束学生，协议和论文一致。旧 checkpoint 直接当多教师，如果类别覆盖不一致，就不是严格 MTD。

放到项目里怎么用

- 正式矩阵不是随便堆方法，而是 sanity、单教师主基线、多教师主基线、分类偏差基线、多教师偏差基线。
- MTD-SSIL 已在官方配置存在，只能作为基线，不能作为创新点。

常见误区

- 误区：教师越多越好。教师质量、覆盖范围、多样性和成本都要检查。
- 误区：KD 一定解决遗忘。KD 的 softmax 范围、温度、样本分布会决定它能看见什么问题。

分层练习：给每个方法写一句话：它用什么额外信息保护旧类，以及它可能解决不了什么。

本课专属任务：本课专属任务：从 LwF 损失写出 teacher 输出、student 输出、temperature 三个角色，说明 teacher 为什么不更新。

放回本项目：LwF 作为 sanity baseline 可跑 1 个种子，用来证明“只靠蒸馏且无重放”在本协议中的表现；不能直接拿不同论文设定的数字硬比较。

练习与自检：LwF 为什么称为 data-free replay-free，但仍需要数据训练？

参考答案：它不保存旧类数据，但仍使用当前的新类数据；“无数据”只指无旧数据。

第 17 课 Replay 与 exemplar

学完这一课，你应该会

- 理解重放如何恢复旧类信号
- 知道类别平衡问题
- 解释 exemplar 选择与预算

为什么有效

把少量旧样本与新样本一起训练，旧类不再永远充当负类。CE 能直接抬高旧样本的正确旧类 logit，并约束特征空间。即使每类只有 20 张，也通常比完全无旧数据更能缓解偏差。

仍需注意

新类有完整数据、旧类只有少量 exemplar，batch 比例和采样策略会影响偏置。exemplar 可随机选，也可按特征均值选代表样本。公平比较必须固定预算与选择规则。

零基础加厚讲解

先修回顾：这一部分学习方法。不要先背名字，先问每个方法用什么信号保护旧知识：输出、特征、样本、教师，还是分类偏差校正。

直觉解释

LwF 用旧模型输出约束学生；Replay 把旧样本混回训练；SS-IL 用 separated softmax 缓解新旧类竞争偏差；PODNet 蒸馏中间特征；MTD 构造多个教师增加蒸馏信息。

方法有效必须同时满足两个条件：训练信号确实能约束学生，协议和论文一致。旧 checkpoint 直接当多教师，如果类别覆盖不一致，就不是严格 MTD。

放到项目里怎么用

- 正式矩阵不是随便堆方法，而是 sanity、单教师主基线、多教师主基线、分类偏差基线、多教师偏差基线。
- MTD-SSIL 已在官方配置存在，只能作为基线，不能作为创新点。

常见误区

- 误区：教师越多越好。教师质量、覆盖范围、多样性和成本都要检查。
- 误区：KD 一定解决遗忘。KD 的 softmax 范围、温度、样本分布会决定它能看见什么问题。

分层练习：给每个方法写一句话：它用什么额外信息保护旧类，以及它可能解决不了什么。

本课专属任务：本课专属任务：对比“无 replay 的 KD”和“有 replay 的 KD”，说明样本分布为什么会影响蒸馏是否有用。

放回本项目：D3 在 D1 基础上加入每类 20 个旧样本重放，旧类平均准确率从 0.18% 提升到 58.73%，是当前最清晰的诊断证据之一。

练习与自检：为什么有 replay 仍可能偏向新类？

参考答案：因为新旧样本数量、训练频率和分布仍不平衡；分类器也可能需要额外校正。

第 18 课 Separated Softmax 与 SS-IL

学完这一课，你应该会

- 理解分组归一化
- 说明它针对哪类偏差
- 知道何时不能断言有效

核心直觉

普通 softmax 让旧类和新类在不平衡数据下直接竞争。Separated softmax 把旧类组和新类组分开处理，减少新类样本对全部旧类的统一压制；SS-IL 将这类分离策略与增量训练结合。

解释边界

它主要缓解分类器层面的偏差，并不保证特征完全不忘。是否有效必须看相同 replay、骨干和训练预算下的 old/new、AIA 与 forgetting，而不是只看一个阶段。

零基础加厚讲解

先修回顾：这一部分学习方法。不要先背名字，先问每个方法用什么信号保护旧知识：输出、特征、样本、教师，还是分类偏差校正。

直觉解释

LwF 用旧模型输出约束学生；Replay 把旧样本混回训练；SS-IL 用 separated softmax 缓解新旧类竞争偏差；PODNet 蒸馏中间特征；MTD 构造多个教师增加蒸馏信息。

方法有效必须同时满足两个条件：训练信号确实能约束学生，协议和论文一致。旧 checkpoint 直接当多教师，如果类别覆盖不一致，就不是严格 MTD。

放到项目里怎么用

- 正式矩阵不是随便堆方法，而是 sanity、单教师主基线、多教师主基线、分类偏差基线、多教师偏差基线。
- MTD-SSIL 已在官方配置存在，只能作为基线，不能作为创新点。

常见误区

- 误区：教师越多越好。教师质量、覆盖范围、多样性和成本都要检查。
- 误区：KD 一定解决遗忘。KD 的 softmax 范围、温度、样本分布会决定它能看见什么问题。

分层练习：给每个方法写一句话：它用什么额外信息保护旧类，以及它可能解决不了什么。

本课专属任务：本课专属任务：用一句话解释 separated softmax：新旧类别不直接在同一个 softmax 里无控制竞争，从而缓解偏向新类。

放回本项目：D4 在 replay 基础上加入 separated softmax，当前 6 epoch 诊断中 old accuracy 58.91%，与 D3 的 58.73% 很接近；短诊断不能夸大 0.18 个百分点差异。

练习与自检：D4 比 D3 高 0.18 个百分点，能否直接说 separated softmax 有效？

参考答案：不能。只有单种子、短训练，差异可能是随机波动；需多种子和完整协议验证。

第 19 课 PODNet：保护表示空间

学完这一课，你应该会

- 理解 pooled feature distillation
- 知道与只蒸馏 logits 的区别
- 说明为何选作主基线

方法直觉

PODNet 不只约束最终 logits，还对中间特征做空间池化蒸馏。它试图保留旧模型在不同层的表示结构，同时配合适合增量分类的分类器与 exemplar 机制。

为什么重要

只冻结或蒸馏分类器无法阻止特征提取器漂移。对中间表示施加约束，能从另一条路径保护旧知识。代价是额外前向、特征存储和超参数。

零基础加厚讲解

先修回顾：这一部分学习方法。不要先背名字，先问每个方法用什么信号保护旧知识：输出、特征、样本、教师，还是分类偏差校正。

直觉解释

LwF 用旧模型输出约束学生；Replay 把旧样本混回训练；SS-IL 用 separated softmax 缓解新旧类竞争偏差；PODNet 蒸馏中间特征；MTD 构造多个教师增加蒸馏信息。

方法有效必须同时满足两个条件：训练信号确实能约束学生，协议和论文一致。旧 checkpoint 直接当多教师，如果类别覆盖不一致，就不是严格 MTD。

放到项目里怎么用

- 正式矩阵不是随便堆方法，而是 sanity、单教师主基线、多教师主基线、分类偏差基线、多教师偏差基线。
- MTD-SSIL 已在官方配置存在，只能作为基线，不能作为创新点。

常见误区

- 误区：教师越多越好。教师质量、覆盖范围、多样性和成本都要检查。
- 误区：KD 一定解决遗忘。KD 的 softmax 范围、温度、样本分布会决定它能看见什么问题。

分层练习：给每个方法写一句话：它用什么额外信息保护旧类，以及它可能解决不了什么。

本课专属任务：本课专属任务：说明 PODNet 为什么不只蒸馏最终 logits，而要约束中间特征或 pooled outputs。

放回本项目：官方 CLearning 中 PODNet 是单教师主基线，目标是在固定 B50-5S 下先复现到论文附近，再比较 MTD-PODNet。

练习与自检：PODNet 比普通 logit KD 多保护了什么？

参考答案：它还约束中间层的空间/通道表示关系，减少旧特征结构漂移。

第 20 课 MTD：真正的多教师蒸馏

学完这一课，你应该会

- 理解多教师为何可能互补
- 知道教师多样性与质量
- 解释聚合与额外成本

多教师不是简单堆模型

Multi-Teacher Distillation 希望不同教师在表示或预测上提供互补信息。有效多样性应由受控机制产生，例如权重置换、特征扰动或不同训练视角，同时教师质量不能太差。

学生如何学习

学生可对多个教师分别计算蒸馏损失，再按权重聚合。若教师高度相同，新增教师只增加成本；若教师互相冲突，错误聚合还会伤害学生。因此必须同时报告性能、方差、参数、显存和训练时间。

零基础加厚讲解

先修回顾：这一部分学习方法。不要先背名字，先问每个方法用什么信号保护旧知识：输出、特征、样本、教师，还是分类偏差校正。

直觉解释

LwF 用旧模型输出约束学生；Replay 把旧样本混回训练；SS-IL 用 separated softmax 缓解新旧类竞争偏差；PODNet 蒸馏中间特征；MTD 构造多个教师增加蒸馏信息。

方法有效必须同时满足两个条件：训练信号确实能约束学生，协议和论文一致。旧 checkpoint 直接当多教师，如果类别覆盖不一致，就不是严格 MTD。

放到项目里怎么用

- 正式矩阵不是随便堆方法，而是 sanity、单教师主基线、多教师主基线、分类偏差基线、多教师偏差基线。
- MTD-SSIL 已在官方配置存在，只能作为基线，不能作为创新点。

常见误区

- 误区：教师越多越好。教师质量、覆盖范围、多样性和成本都要检查。
- 误区：KD 一定解决遗忘。KD 的 softmax 范围、温度、样本分布会决定它能看见什么问题。

分层练习：给每个方法写一句话：它用什么额外信息保护旧类，以及它可能解决不了什么。

本课专属任务：本课专属任务：列出真正 MTD 的三个条件：教师覆盖一致、教师有受控多样性、学生从多个教师得到有效梯度。

放回本项目：正式矩阵使用原始教师加 1 个额外教师，并比较 PODNet/MTD-PODNet 与 SS-IL/MTD-SSIL 的 2×2 关系。

练习与自检：教师数量从 1 变 2，为什么不能自动称为更好？

参考答案：第二个教师可能重复、较差或与第一个冲突；只有稳定超过随机波动且成本合理才有价值。

第 21 课 历史 checkpoint 为什么不等于 MTD

学完这一课，你应该会

- 检查教师类别覆盖
- 区分时间差异与受控多样性
- 发现伪多教师实现

覆盖问题

10 类 checkpoint 只包含 10 类输出，11 类 checkpoint 多一个类别。把它们同时称为教师时，它们并未在同一知识空间内提供平等意见；10 类教师无法保护第 11 类。

多样性问题

历史阶段不同只是训练时间与类别覆盖不同，不等于论文设计的多样化教师。还要检查损失项是否依赖学生、参数是否传入、教师是否 eval/frozen、输出维度如何对齐。

零基础加厚讲解

先修回顾：这一部分学习方法。不要先背名字，先问每个方法用什么信号保护旧知识：输出、特征、样本、教师，还是分类偏差校正。

直觉解释

LwF 用旧模型输出约束学生；Replay 把旧样本混回训练；SS-IL 用 separated softmax 缓解新旧类竞争偏差；PODNet 蒸馏中间特征；MTD 构造多个教师增加蒸馏信息。

方法有效必须同时满足两个条件：训练信号确实能约束学生，协议和论文一致。旧 checkpoint 直接当多教师，如果类别覆盖不一致，就不是严格 MTD。

放到项目里怎么用

- 正式矩阵不是随便堆方法，而是 sanity、单教师主基线、多教师主基线、分类偏差基线、多教师偏差基线。
- MTD-SSIL 已在官方配置存在，只能作为基线，不能作为创新点。

常见误区

- 误区：教师越多越好。教师质量、覆盖范围、多样性和成本都要检查。
- 误区：KD 一定解决遗忘。KD 的 softmax 范围、温度、样本分布会决定它能看见什么问题。

分层练习：给每个方法写一句话：它用什么额外信息保护旧类，以及它可能解决不了什么。

本课专属任务：本课专属任务：解释为什么 10 类 checkpoint 不能保护第 11 类 bus/train 之后的新知识。

放回本项目：交接审计发现 10 类和 11 类检查点不是真正 MTD 教师，且双教师代码存在缺参与无学生梯度项，因此旧双教师结果不可作为正式证据。

练习与自检：判断一个“多教师”实现是否可信，至少检查哪三项？

参考答案：教师类别覆盖一致性、教师多样性来源、每个蒸馏项是否对学生产生非零梯度；还应检查冻结、eval 和输出对齐。

第五部分 读懂本项目与类别替换故障

第 22 课 如何读交接文档

学完这一课，你应该会

- 从事实、解释、待办三层拆解文档
- 识别不可比实验
- 建立证据索引

三层阅读法

第一层摘录可验证事实：数据路径、脚本、checkpoint、准确率、改动时间。第二层标注文档作者的解释，它可能正确也可能只是猜测。第三层列出仍需实验回答的问题。不要把三层混在一起。

时间线与变更集

将每次实验的代码、数据、优化器、epoch、种子放在同一行。如果一次同时修改 RGB/BGR、优化器和训练轮次，就只能说“组合改动后结果变化”，不能归因给某一项。

零基础加厚讲解

先修回顾：这一部分回到交接项目。目标不是替旧代码辩护，而是把故障链条拆成能被实验证明或否定的命题。

直觉解释

bus -> train -> wolf 的核心不是某个类别特殊，而是增量训练中最新类获得过强决策优势。若只有新类数据和新类 CE，旧类 logit 整体被压低，最新类就会吸走旧样本。

旧类内部 KD 只比较旧类之间的相对分布。给所有旧类 logits 同时减一个常数，旧类内部 softmax 基本不变，所以它看不见旧类整体下移。

放到项目里怎么用

- D0-D4 的数字要会背后的含义：D0/D1 old 接近 0，D3/D4 old 约 58，说明 replay 显著缓解极端替换。
- KD common shift delta 约 $1.19e-7$ ，是数值证据：旧类内部 KD 对共同平移几乎不敏感。

常见误区

- 误区：看到 D4 比 D3 高一点就说 separated softmax 有效。当前差异很小，不能超过证据边界。
- 误区：把单种子 6 epoch 诊断当正式性能结论。它只能定位机制。

分层练习：用 30 秒解释：为什么旧类 logits 同时下移会让 CIL 失败，而旧类内部 KD 不报警。

本课专属任务：本课专属任务：把交接文档中的每条说法标成事实、解释、待验证三类，避免把猜测当证据。

放回本项目：交接文档是故障线索来源，不是自动可信的实验报告。新审计表把旧实现、标准实现、可能影响和验证状态并列。

练习与自检：文档写“换 Adam 后准确率提高”，但同次还修了通道并加了 epoch。正确结论是什么？

参考答案：只能说这些组合改动后提高，无法单独归因给 Adam；需要受控消融。

第 23 课 如何读旧 VGG 增量代码

学完这一课，你应该会

- 追踪数据、模型、损失、优化器
- 检查输出层扩展
- 检查评估与 checkpoint 规则

四条主线

数据线：每阶段训练集到底含哪些类；模型线：输出层从 C 扩展到 C+1 时如何初始化和复制；损失线：CE、KD 和多教师项分别依赖谁；优化线：哪些参数在 optimizer 中、预热阶段冻结了谁。

高风险代码

重新创建 nn.Linear 后若未复制旧行，等于丢掉分类器知识。复制后若整个层在仅新类数据上更新，旧行仍会被 CE 压低。评估若用测试集挑 epoch，会产生泄漏。

零基础加厚讲解

先修回顾：这一部分回到交接项目。目标不是替旧代码辩护，而是把故障链条拆成能被实验证明或否定的命题。

直觉解释

bus -> train -> wolf 的核心不是某个类别特殊，而是增量训练中最新类获得过强决策优势。若只有新类数据和新类 CE，旧类 logit 整体被压低，最新类就会吸走旧样本。

旧类内部 KD 只比较旧类之间的相对分布。给所有旧类 logits 同时减一个常数，旧类内部 softmax 基本不变，所以它看不见旧类整体下移。

放到项目里怎么用

- D0-D4 的数字要会背后的含义：D0/D1 old 接近 0，D3/D4 old 约 58，说明 replay 显著缓解极端替换。
- KD common shift delta 约 1.19e-7，是数值证据：旧类内部 KD 对共同平移几乎不敏感。

常见误区

- 误区：看到 D4 比 D3 高一点就说 separated softmax 有效。当前差异很小，不能超过证据边界。
- 误区：把单种子 6 epoch 诊断当正式性能结论。它只能定位机制。

分层练习：用 30 秒解释：为什么旧类 logits 同时下移会让 CIL 失败，而旧类内部 KD 不报警。

本课专属任务：本课专属任务：定位旧实现的分类器扩展代码，检查是否复制旧权重和偏置，以及 optimizer 是否更新旧行。

放回本项目：已定位 train_CIL12.py 的损失和 train_CIL12_10p11.py 的双教师逻辑；正式项目不直接修补旧目录，而是在隔离工作区重建可审计流程。

练习与自检：输出层从 11 扩到 12 类时，最基本的权重继承应怎么做？

参考答案：新建 12 行分类器，将前 11 行权重和偏置复制旧模型，第 12 行单独初始化；再明确决定哪些行可更新。

第 24 课 bus -> train -> wolf 类别替换

学完这一课，你应该会

- 复述现象
- 用 CE 与 KD 联合解释
- 把准确率突变连接到 logit gap

现象

交接记录中，加入 train 后 bus 从约 0.57 降到 0、train 从 0 升到约 0.83；下一阶段加入 wolf，train 从约 0.81 降到 0、wolf 升到约 0.82。模式不是随机忘几个类，而是最新类持续替换上一个新类。

机制链

仅新类训练 -> CE 抬高新类并压低所有旧类 -> 旧类内部 KD 只保持相对顺序、看不到共同下移 -> new-old logit gap 越过决策边界 -> 旧样本被预测为最新类 -> 旧类准确率归零。输出层重初始化或整层更新会进一步放大问题。

零基础加厚讲解

先修回顾：这一部分回到交接项目。目标不是替旧代码辩护，而是把故障链条拆成能被实验证明或否定的命题。

直觉解释

bus -> train -> wolf 的核心不是某个类别特殊，而是增量训练中最新类获得过强决策优势。若只有新类数据和新类 CE，旧类 logit 整体被压低，最新类就会吸走旧样本。

旧类内部 KD 只比较旧类之间的相对分布。给所有旧类 logits 同时减一个常数，旧类内部 softmax 基本不变，所以它看不见旧类整体下移。

放到项目里怎么用

- D0-D4 的数字要会背后的含义：D0/D1 old 接近 0，D3/D4 old 约 58，说明 replay 显著缓解极端替换。
- KD common shift delta 约 $1.19e-7$ ，是数值证据：旧类内部 KD 对共同平移几乎不敏感。

常见误区

- 误区：看到 D4 比 D3 高一点就说 separated softmax 有效。当前差异很小，不能超过证据边界。
- 误区：把单种子 6 epoch 诊断当正式性能结论。它只能定位机制。

分层练习：用 30 秒解释：为什么旧类 logits 同时下移会让 CIL 失败，而旧类内部 KD 不报警。

本课专属任务：本课专属任务：手算 $\text{softmax}(z_{\text{old}}+a)$ 的不变性，说明旧类内部 KD 为什么看不到旧类整体下移。

放回本项目：D0 实际诊断在 6 epoch 后 bus=0%、train=100%、old=0%、gap=26.99，复现了极端类别替换。

练习与自检：请用一条因果链解释为什么旧类 KD 很小但旧类准确率仍可归零。

参考答案：旧类 logits 共同下移不改变旧类内部 softmax，所以 KD 很小；但单头分类还要与新类 logit 比较，新类超过全部旧类后旧样本被统一抢走。

第 25 课 受控实验与因果归因

学完这一课，你应该会

- 理解一次只改一个关键因素
- 区分对照、消融和复现
- 知道随机种子的作用

对照逻辑

D0 是当前实现；D1 只纠正旧权重继承；D2 在 D1 上冻结旧分类器行；D3 在 D1 上加入 replay；D4 在 D3 上加入 separated softmax。相邻比较尽量对应一个机制。

注意交互

现实中因素会交互，因此还要明确基准。D2 与 D3 都从 D1 出发，不应把 D3-D2 简单解释为 replay 的纯效应。正式研究还需多种子、完整训练与置信区间。

零基础加厚讲解

先修回顾：这一部分回到交接项目。目标不是替旧代码辩护，而是把故障链条拆成能被实验证明或否定的命题。

直觉解释

bus -> train -> wolf 的核心不是某个类别特殊，而是增量训练中最新类获得过强决策优势。若只有新类数据和新类 CE，旧类 logit 整体被压低，最新类就会吸走旧样本。

旧类内部 KD 只比较旧类之间的相对分布。给所有旧类 logits 同时减一个常数，旧类内部 softmax 基本不变，所以它看不见旧类整体下移。

放到项目里怎么用

- D0-D4 的数字要会背后的含义：D0/D1 old 接近 0，D3/D4 old 约 58，说明 replay 显著缓解极端替换。
- KD common shift delta 约 $1.19e-7$ ，是数值证据：旧类内部 KD 对共同平移几乎不敏感。

常见误区

- 误区：看到 D4 比 D3 高一点就说 separated softmax 有效。当前差异很小，不能超过证据边界。
- 误区：把单种子 6 epoch 诊断当正式性能结论。它只能定位机制。

分层练习：用 30 秒解释：为什么旧类 logits 同时下移会让 CIL 失败，而旧类内部 KD 不报警。

本课专属任务：本课专属任务：用 D0、D1、D2 的差异解释“初始化错误”和“分类器漂移”分别贡献什么。

放回本项目：当前诊断固定种子、只跑 6 epoch，目的不是给最终 SOTA 排名，而是快速验证故障机制。

练习与自检：D4-D3 可以初步估计什么？

参考答案：在相同 replay 基础上加入 separated softmax 的增量影响；但单种子短训练只能作诊断线索。

第 26 课 D0-D4：怎样读真实诊断结果

学完这一课，你应该会

- 读懂五组对照
- 判断哪些结论已成立
- 识别不能下的结论

真实结果

下表来自当前项目 raw 日志自动汇总，单位为百分比；logit gap 越大表示新类相对旧类越占优势。

零基础加厚讲解

先修回顾：这一部分回到交接项目。目标不是替旧代码辩护，而是把故障链条拆成能被实验证明或否定的命题。

直觉解释

bus -> train -> wolf 的核心不是某个类别特殊，而是增量训练中最新类获得过强决策优势。若只有新类数据和新类 CE，旧类 logit 整体被压低，最新类就会吸走旧样本。

旧类内部 KD 只比较旧类之间的相对分布。给所有旧类 logits 同时减一个常数，旧类内部 softmax 基本不变，所以它看不见旧类整体下移。

放到项目里怎么用

- D0-D4 的数字要会背后的含义：D0/D1 old 接近 0，D3/D4 old 约 58，说明 replay 显著缓解极端替换。
- KD common shift delta 约 1.19e-7，是数值证据：旧类内部 KD 对共同平移几乎不敏感。

常见误区

- 误区：看到 D4 比 D3 高一点就说 separated softmax 有效。当前差异很小，不能超过证据边界。
- 误区：把单种子 6 epoch 诊断当正式性能结论。它只能定位机制。

分层练习：用 30 秒解释：为什么旧类 logits 同时下移会让 CIL 失败，而旧类内部 KD 不报警。

本课专属任务：本课专属任务：用 D3、D4 的 old accuracy 说明 replay 明显缓解替换，但 separated softmax 的额外收益仍需正式验证。

放回本项目：D0-D4 的证据支持“仅修继承或冻结不足，replay 显著缓解极端替换”；尚不能支持“D4 一定优于 D3”。

练习与自检：哪一项改动在当前诊断中带来最大 old accuracy 改善？

参考答案：从 D1 到 D3 加入每类 20 个旧样本重放，old accuracy 从 0.18% 到 58.73%，改善最大。

设置	bus acc	train acc	old acc	logit gap	类别替换
D0 当前实现	0.00	100.00	0.00	26.99	是
D1 正确复制旧权重	0.00	100.00	0.18	19.37	是
D2 冻结旧分类器行	0.00	100.00	1.36	21.60	是
D3 D1 + replay	10.00	94.00	58.73	13.53	显著缓解
D4 D3 + separated softmax	10.00	94.00	58.91	13.78	显著缓解

表注：固定种子、11->12 类、前 6 epoch 的诊断实验。表中数字不等于标准 B50-5S 主结果。

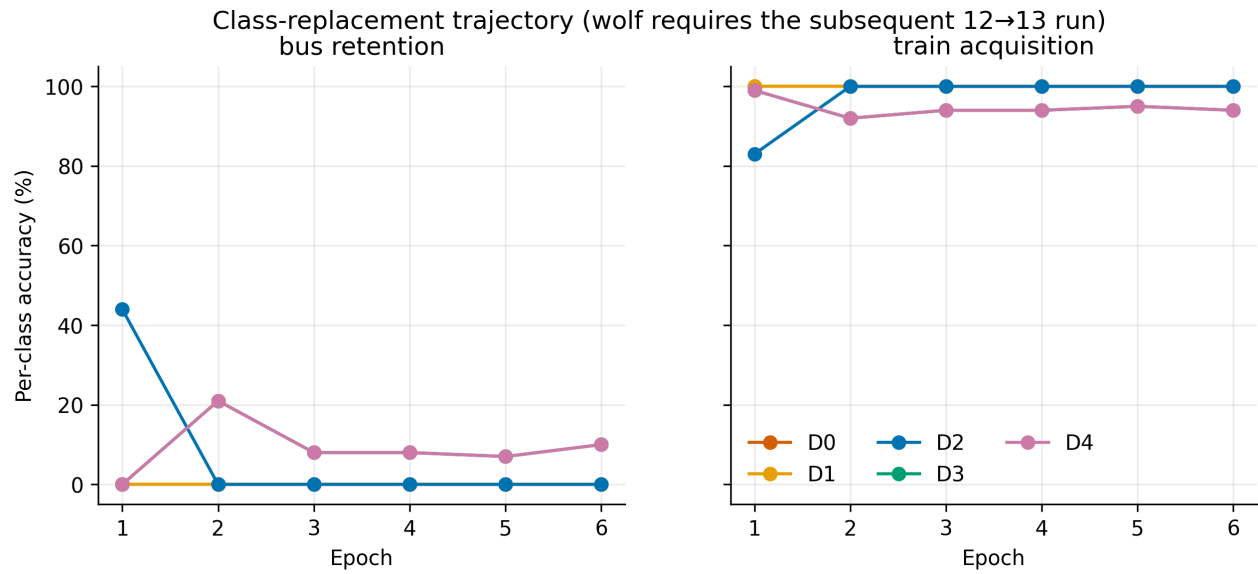


图 1 D0-D4 中 bus/train 的逐 epoch 准确率轨迹。

第 27 课 诊断日志：不只记准确率

学完这一课，你应该会

- 理解每个诊断字段
- 从梯度和权重寻找机制
- 知道日志为何必须结构化

每 epoch 应记录

bus、train、old 平均准确率；old/new 平均 logits 与 gap；各分类器行权重范数和偏置；CE/KD 对旧类行、新类行的梯度大小与方向；bus 错误流向哪个类别。

如何串证据

若 gap 上升与 bus 准确率下降同步，混淆又集中流向 train，同时 CE 梯度持续压旧抬新，而 KD 对共同平移不敏感，就形成从优化到决策再到错误的完整证据链。

零基础加厚讲解

先修回顾：这一部分回到交接项目。目标不是替旧代码辩护，而是把故障链条拆成能被实验证明或否定的命题。

直觉解释

bus -> train -> wolf 的核心不是某个类别特殊，而是增量训练中最新类获得过强决策优势。若只有新类数据和新类 CE，旧类 logit 整体被压低，最新类就会吸走旧样本。

旧类内部 KD 只比较旧类之间的相对分布。给所有旧类 logits 同时减一个常数，旧类内部 softmax 基本不变，所以它看不见旧类整体下移。

放到项目里怎么用

- D0-D4 的数字要会背后的含义：D0/D1 old 接近 0，D3/D4 old 约 58，说明 replay 显著缓解极端替换。
- KD common shift delta 约 1.19e-7，是数值证据：旧类内部 KD 对共同平移几乎不敏感。

常见误区

- 误区：看到 D4 比 D3 高一点就说 separated softmax 有效。当前差异很小，不能超过证据边界。
- 误区：把单种子 6 epoch 诊断当正式性能结论。它只能定位机制。

分层练习：用 30 秒解释：为什么旧类 logits 同时下移会让 CIL 失败，而旧类内部 KD 不报警。

本课专属任务：本课专属任务：把图 1 和图 2 对齐阅读，指出准确率归零是否与 logit gap 越界同步。

放回本项目：raw JSON 是不可手改的事实源；tidy CSV、三线表和图片都从 raw 自动生成，避免手抄数字互相矛盾。

练习与自检：单看 bus 准确率为 0，为什么还不能确定它被 train 替换？

参考答案：bus 也可能被多个旧类误分。必须看混淆去向、train logit 和 old/new gap 才能确认“被最新类替换”。

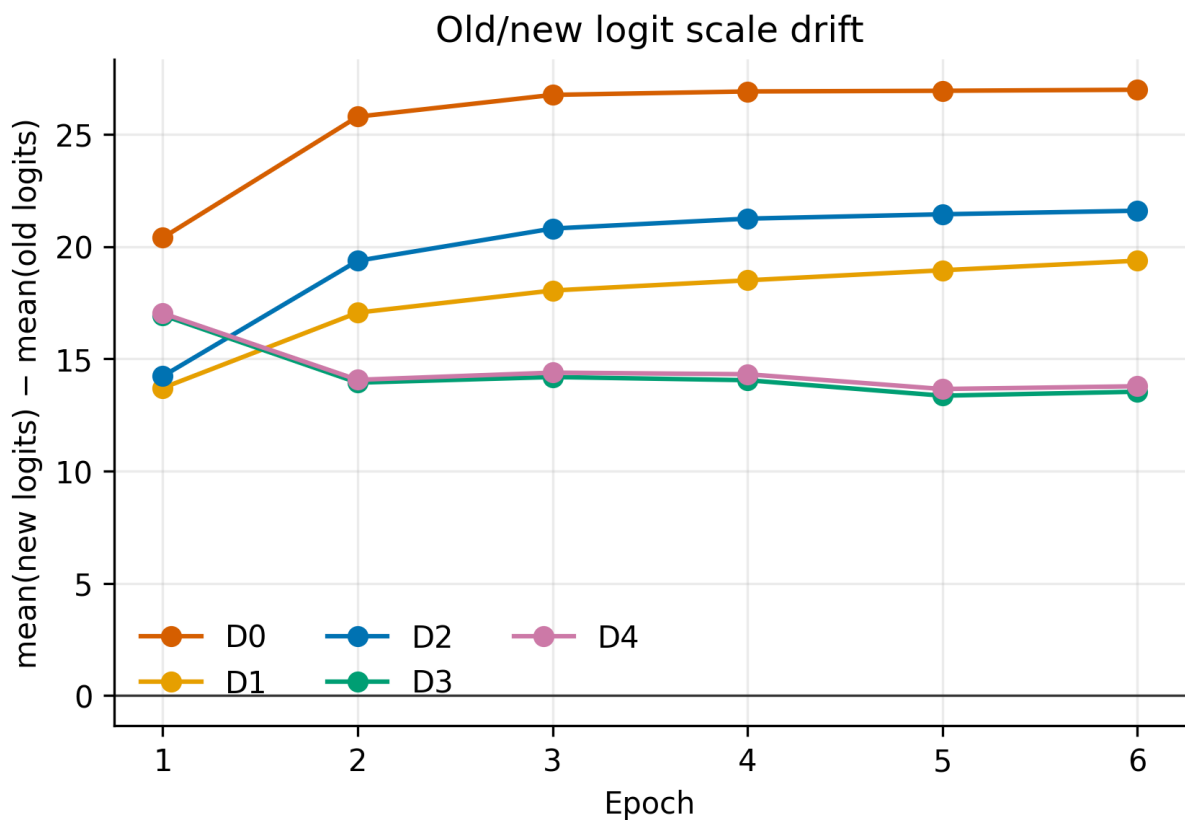


图 2 old/new logit gap 轨迹；需与图 1 同步阅读。

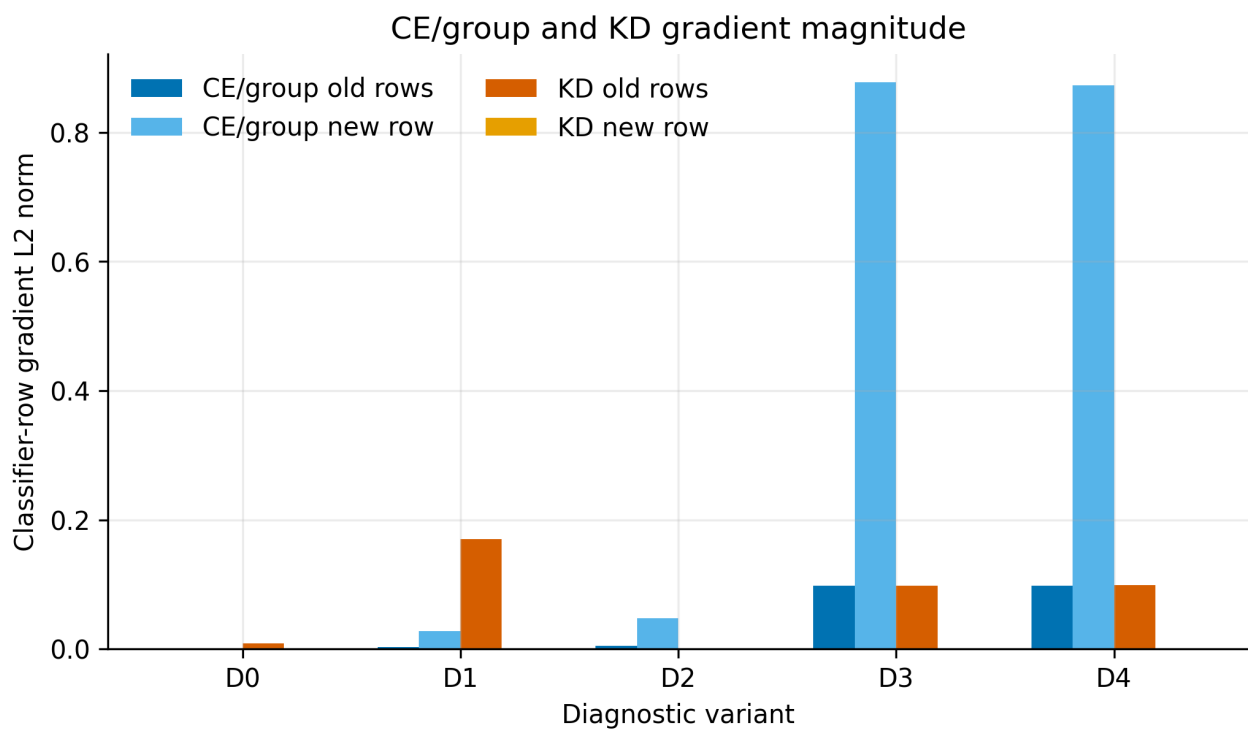


图 3 CE 与 KD 对旧/新输出的梯度诊断。

第六部分 指标、随机性与可信结论

第 28 课 overall、old、new 与逐类准确率

学完这一课，你应该会

- 会计算 accuracy
- 理解宏平均
- 避免类别数量掩盖偏差

基本计算

accuracy = 正确样本数/总样本数。overall 对所有已学类别统计；old 只统计进入本阶段前的类；new 只统计本阶段新增类。逐类准确率先对每类单独计算，可观察特定类是否归零。

宏平均与样本平均

若每类测试样本相同，逐类宏平均与总体样本准确率接近；若样本数不平衡，两者不同。报告时应说明采用哪种。类增量研究常强调按类别或任务公平衡量。

零基础加厚讲解

先修回顾：这一部分回到交接项目。目标不是替旧代码辩护，而是把故障链条拆成能被实验证明或否定的命题。

直觉解释

bus -> train -> wolf 的核心不是某个类别特殊，而是增量训练中最新类获得过强决策优势。若只有新类数据和新类 CE，旧类 logit 整体被压低，最新类就会吸走旧样本。

旧类内部 KD 只比较旧类之间的相对分布。给所有旧类 logits 同时减一个常数，旧类内部 softmax 基本不变，所以它看不见旧类整体下移。

放到项目里怎么用

- D0-D4 的数字要会背后的含义：D0/D1 old 接近 0，D3/D4 old 约 58，说明 replay 显著缓解极端替换。
- KD common shift delta 约 $1.19e-7$ ，是数值证据：旧类内部 KD 对共同平移几乎不敏感。

常见误区

- 误区：看到 D4 比 D3 高一点就说 separated softmax 有效。当前差异很小，不能超过证据边界。
- 误区：把单种子 6 epoch 诊断当正式性能结论。它只能定位机制。

分层练习：用 30 秒解释：为什么旧类 logits 同时下移会让 CIL 失败，而旧类内部 KD 不报警。

本课专属任务：本课专属任务：解释双教师损失中无学生梯度项为什么不能训练学生，并给出检查 grad 的最小流程。

放回本项目：表 1 同时列 bus、train、old 和 gap，就是为了防止 overall 把极端替换平均掉。

练习与自检：90 个旧类各 50%，10 个新类各 100%，若每类样本相同，overall 是多少？

参考答案： $(90 \times 50\% + 10 \times 100\%) / 100 = 55\%$ 。它看似尚可，但新旧差距很大。

第 29 课 AA、AIA 与 forgetting

学完这一课，你应该会

- 会计算阶段准确率
- 理解跨阶段平均
- 会解释遗忘

定义

在阶段 t 结束后，对全部已学类别的准确率记为 AA_t 。AIA (Average Incremental Accuracy) 通常是所有阶段 AA_t 的算术平均： $(1/(T+1))\sum_t AA_t$ 。final AA 是最后阶段准确率。

遗忘

对某个旧任务 k ，可用历史最好准确率减去当前准确率表示遗忘，再对旧任务平均。不同论文定义略有差异，必须在表注或代码中固定一个公式。遗忘越低通常越好，但若模型从一开始就没学会，遗忘也可能虚假地低。

零基础加厚讲解

先修回顾：指标是实验语言。没有统一指标，表格只是数字堆；有统一指标，才能判断方法是否真正优于随机波动。

直觉解释

AA 是某阶段已学类别平均准确率；AIA 是各阶段 AA 的平均；forgetting 衡量历史最好到当前的下降。均值±标准差告诉你不同 seed 下是否稳定。

一个方法 AIA 高但 forgetting 也高，可能只是新类学得不好；old/new accuracy 和稳定性-可塑性散点图可以拆开来。

放到项目里怎么用

- 正式主表必须报告 PODNet、MTD-PODNet、SS-IL、MTD-SSIL 的三种子均值±标准差。
- 若 MTD 提升小于标准差，汇报时不能说“有效提升”，只能说“当前未观察到稳定增益”。

常见误区

- 误区：手工抄数字进表。项目要求所有表图从 raw JSON 自动生成。
- 误区：缺一个 seed 也写均值±标准差。生成脚本应拒绝。

分层练习：手算一次 AIA：若 6 个阶段 AA 为 70、65、60、58、55、52，则 AIA 是它们的算术平均。

本课专属任务：本课专属任务：给 AA、AIA、forgetting 各举一个数字例子，保证能手算而不是只会读脚本输出。

放回本项目：项目中所有表只调用同一个 metrics 函数计算 AIA 与 forgetting；图终点必须与主表 final AA 一致。

练习与自检：阶段 AA 为 70、65、60，AIA 是多少？

参考答案： $(70+65+60)/3 = 65$ 。

第 30 课 随机种子、均值与标准差

学完这一课，你应该会

- 理解训练随机性来源
- 会读 $\text{mean} \pm \text{std}$
- 知道显著改善的最低判断

随机性

权重初始化、数据打乱、数据增强、exemplar 选择和 GPU 算法都可能改变结果。固定种子能复现某次运行，但不能证明方法对不同随机条件稳定。

均值与标准差

三种子结果写为均值 $\pm$ 标准差。均值代表典型水平，标准差反映波动。若 MTD 的平均提升不超过双方波动范围，应谨慎表述为“未观察到稳定提升”，而不是宣称有效。三种子只是最低限度，严格统计还需更多运行或配对检验。

零基础加厚讲解

先修回顾：指标是实验语言。没有统一指标，表格只是数字堆；有统一指标，才能判断方法是否真正优于随机波动。

直觉解释

AA 是某阶段已学类别平均准确率；AIA 是各阶段 AA 的平均；forgetting 衡量历史最好到当前的下降。均值 $\pm$ 标准差告诉你不同 seed 下是否稳定。

一个方法 AIA 高但 forgetting 也高，可能只是新类学得好；old/new accuracy 和稳定性-可塑性散点图可以拆开来。

放到项目里怎么用

- 正式主表必须报告 PODNet、MTD-PODNet、SS-IL、MTD-SSIL 的三种子均值 $\pm$ 标准差。
- 若 MTD 提升小于标准差，汇报时不能说“有效提升”，只能说“当前未观察到稳定增益”。

常见误区

- 误区：手工抄数字进表。项目要求所有表图从 raw JSON 自动生成。
- 误区：缺一个 seed 也写均值 $\pm$ 标准差。生成脚本应拒绝。

分层练习：手算一次 AIA：若 6 个阶段 AA 为 70、65、60、58、55、52，则 AIA 是它们的算术平均。

本课专属任务：本课专属任务：说明为什么三种子要报告 $\text{mean} \pm \text{std}$ ，并写出“提升小于标准差”时的谨慎表述。

放回本项目：主基线 PODNet、MTD-PODNet、SS-IL、MTD-SSIL 使用 1993/1994/1995 三个种子；sanity 方法只跑 1 种子且不作强结论。

练习与自检：方法 A 为 65.0 ± 1.5 ，B 为 65.4 ± 1.7 ，能否说 B 明显更好？

参考答案：不能。0.4 的均值差小于波动，需更多证据；可描述为结果相近。

第七部分 工程：配置、日志与可复现性

第 31 课 YAML 配置怎么读

学完这一课，你应该会

- 找到实验关键字段
- 区分方法配置与运行覆盖
- 避免隐式默认值

关键字段

配置通常包括数据集、类别顺序、初始类数、增量数、骨干、batch、epoch、优化器、学习率、memory、蒸馏权重、种子和输出目录。读配置时先把这些字段做成一页摘要。

默认值风险

同名字段可能在基础配置、方法配置和命令行覆盖中层层合并。最终运行配置必须完整保存到 raw 目录，不能只保存手写 YAML，否则之后不知道实际默认值。

零基础加厚讲解

先修回顾：指标是实验语言。没有统一指标，表格只是数字堆；有统一指标，才能判断方法是否真正优于随机波动。

直觉解释

AA 是某阶段已学类别平均准确率；AIA 是各阶段 AA 的平均；forgetting 衡量历史最好到当前的下降。均值±标准差告诉你不同 seed 下是否稳定。

一个方法 AIA 高但 forgetting 也高，可能只是新类学得好；old/new accuracy 和稳定性-可塑性散点图可以拆开来。

放到项目里怎么用

- 正式主表必须报告 PODNet、MTD-PODNet、SS-IL、MTD-SSIL 的三种子均值±标准差。
- 若 MTD 提升小于标准差，汇报时不能说“有效提升”，只能说“当前未观察到稳定增益”。

常见误区

- 误区：手工抄数字进表。项目要求所有表图从 raw JSON 自动生成。
- 误区：缺一个 seed 也写均值±标准差。生成脚本应拒绝。

分层练习：手算一次 AIA：若 6 个阶段 AA 为 70、65、60、58、55、52，则 AIA 是它们的算术平均。

本课专属任务：本课专属任务：为主结果表写表下注明：CIFAR-100、B50-5S、memory=20/class、seeds=1993/1994/1995。

放回本项目：configs/official 下的 B50-5S 配置来自固定 CLearning 提交；运行时还应把解析后的最终配置复制进每次结果目录。

练习与自检：为什么只保存命令行不够？

参考答案：命令行可能依赖配置文件和代码默认值；未来版本变化后无法还原实际参数。

第 32 课 Checkpoint、环境与运行记录

学完这一课，你应该会

- 知道 checkpoint 应含什么
- 记录软件与硬件
- 理解断点续训和最佳模型的区别

checkpoint

至少保存模型参数、优化器状态、学习率调度器状态、当前阶段/epoch、随机数状态和配置。断点续训需要优化器等完整状态；仅推理可只保模型。

环境记录

记录 Python、PyTorch、CUDA、GPU、依赖版本、Git commit、数据校验信息和运行命令。相同代码在不同依赖下可能出现 API 或随机性差异。

零基础加厚讲解

先修回顾：指标是实验语言。没有统一指标，表格只是数字堆；有统一指标，才能判断方法是否真正优于随机波动。

直觉解释

AA 是某阶段已学类别平均准确率；AIA 是各阶段 AA 的平均；forgetting 衡量历史最好到当前的下降。均值±标准差告诉你不同 seed 下是否稳定。

一个方法 AIA 高但 forgetting 也高，可能只是新类学得不好；old/new accuracy 和稳定性-可塑性散点图可以拆开来。

放到项目里怎么用

- 正式主表必须报告 PODNet、MTD-PODNet、SS-IL、MTD-SSIL 的三种子均值±标准差。
- 若 MTD 提升小于标准差，汇报时不能说“有效提升”，只能说“当前未观察到稳定增益”。

常见误区

- 误区：手工抄数字进表。项目要求所有表图从 raw JSON 自动生成。
- 误区：缺一个 seed 也写均值±标准差。生成脚本应拒绝。

分层练习：手算一次 AIA：若 6 个阶段 AA 为 70、65、60、58、55、52，则 AIA 是它们的算术平均。

本课专属任务：本课专属任务：解释 old accuracy 与 new accuracy 的二维散点图如何反映稳定性和可塑性。

放回本项目：当前正式环境目标是 Python 3.12、PyTorch 2.5.1+cu121、RTX 4060 8GB；CLearning 固定 commit ce0789a40bda9e566a1e0432d3ac320937ca48f0。依赖 continuum==1.2.4 尚未安装成功，所以正式跑数仍待解锁。

练习与自检：为什么不能把“代码已克隆”写成“正式实验已复现”？

参考答案：代码存在不等于依赖可运行、数据正确、训练完成或指标吻合；复现必须有成功日志与结果核验。

第 33 课 raw -> tidy -> tables/figures

学完这一课，你应该会

- 理解单一事实源
- 知道 schema 校验
- 防止手工表图错误

数据流水线

训练每次输出 raw JSON，包含方法、种子、场景、task、AA、old/new、逐类准确率、混淆矩阵、logits、效率等。清洗脚本把不同运行拼成 tidy CSV，一行代表一个方法-种子-阶段。统计脚本再计算均值和标准差。

校验

生成前检查任务数、类别数、种子、缺失值和单位；生成后检查 AIA 等于阶段平均、曲线终点等于 final AA、同一指标只由一个函数计算、缺 seed 时拒绝输出 mean±std。

零基础加厚讲解

先修回顾：指标是实验语言。没有统一指标，表格只是数字堆；有统一指标，才能判断方法是否真正优于随机波动。

直觉解释

AA 是某阶段已学类别平均准确率；AIA 是各阶段 AA 的平均；forgetting 衡量历史最好到当前的下降。均值±标准差告诉你不同 seed 下是否稳定。

一个方法 AIA 高但 forgetting 也高，可能只是新类学得不好；old/new accuracy 和稳定性-可塑性散点图可以拆开来。

放到项目里怎么用

- 正式主表必须报告 PODNet、MTD-PODNet、SS-IL、MTD-SSIL 的三种子均值±标准差。
- 若 MTD 提升小于标准差，汇报时不能说“有效提升”，只能说“当前未观察到稳定增益”。

常见误区

- 误区：手工抄数字进表。项目要求所有表图从 raw JSON 自动生成。
- 误区：缺一个 seed 也写均值±标准差。生成脚本应拒绝。

分层练习：手算一次 AIA：若 6 个阶段 AA 为 70、65、60、58、55、52，则 AIA 是它们的算术平均。

本课专属任务：本课专属任务：把训练时间、峰值显存、额外教师参数和 AIA 放在一起，判断多教师是否值得成本。

放回本项目：results/raw 不可手改；results/tidy、tables、figures、summary 都可由脚本重建。三线表 Excel 已按这一原则生成。

练习与自检：为什么最终 Excel 里的数字不允许手动改？

参考答案：手改会断开与原始日志的可追溯关系，并可能造成表、图、摘要之间不一致。

第八部分 三线表与图表解读

第 34 课 三线表：规则与研究问题

学完这一课，你应该会

- 识别顶线、表头线、底线
- 正确使用粗体和下划线
- 让每张表回答一个问题

版式

三线表不使用竖线和内部网格。顶线/底线较粗，表头分隔线较细；方法左对齐，数值按小数点对齐；准确率两位小数、时间一位；三种子写均值 ± 标准差；最优粗体、次优下划线。

科学约束

不同协议不能混在同一排名。表注必须写数据集、任务划分、memory、种子数和单位。表 1 回答故障机制，表 2 回答主性能，表 3 回答阶段趋势，表 4 回答多教师与偏差校正的二因素关系，表 5 回答成本，表 6 回答代码可信性。

零基础加厚讲解

先修回顾：工程部分的目标是可复现。你要让未来任何人能从配置、日志、脚本重新生成同一张表。

直觉解释

raw 日志是事实源，tidy CSV 是统计入口，tables/figures 是展示结果。展示层不能手改数字，否则证据链断掉。可复现不只是固定 seed，还包括固定代码 commit、依赖版本、数据处理、checkpoint 选择规则、输出目录和校验脚本。

放到项目里怎么用

- 当前官方 CLearning 固定在 commit ce0789a40...48f0；Python 3.12 下 continuum==1.2.4 安装阻塞，所以正式 B50-5S 结果不能伪造。
- 每次运行至少记录 method、seed、scenario、task_id、AA/AIA/forgetting、per-class accuracy、confusion matrix、logit gap、时间和显存。

常见误区

- 误区：跑通一次就算复现。复现还要和论文 AIA 偏差不超过预设阈值，并能解释差异。
- 误区：测试集选最佳 epoch。固定 epoch 或验证集规则必须在训练前写清楚。

分层练习：从一张主表反查：表格数字 -> tidy CSV 行 -> raw JSON -> 配置 -> 代码 commit。

本课专属任务：本课专属任务：写出 raw -> tidy -> tables/figures -> summary 的单向证据链，并说明为什么最终表不能手改。

放回本项目：当前表 1 和表 6 可填真实诊断/审计；表 2-5 的正式数字在三种子 B50-5S 完成前应保持“待实验”。

练习与自检：为什么“空着待实验”比填论文数字更好？

参考答案：论文数字可能来自不同环境或协议，不能冒充本地复现；明确缺口保持证据边界。

第 35 课 增量准确率曲线与遗忘曲线

学完这一课，你应该会

- 读懂横纵轴
- 结合均值线与标准差阴影
- 区分学新类与保旧类

增量曲线

横轴是 Task 0 到 Task 5，纵轴是 AA。曲线越高越好，但要观察优势是否持续；只在早期领先、后期跌落的策略可能不抗遗忘。三种子用均值实线和标准差阴影。

遗忘曲线

遗忘曲线展示旧任务从历史最好到当前的下降。它与 AA 互补：高 AA 可能来自新类很强，低 forgetting 可能来自旧任务一开始很弱。应同时看 old/new 与最终准确率。

零基础加厚讲解

先修回顾：工程部分的目标是可复现。你要让未来任何人能从配置、日志、脚本重新生成同一张表。

直觉解释

raw 日志是事实源，tidy CSV 是统计入口，tables/figures 是展示结果。展示层不能手改数字，否则证据链断掉。

可复现不只是固定 seed，还包括固定代码 commit、依赖版本、数据处理、checkpoint 选择规则、输出目录和校验脚本。

放到项目里怎么用

- 当前官方 CLearning 固定在 commit ce0789a40...48f0；Python 3.12 下 continuum==1.2.4 安装阻塞，所以正式 B50-5S 结果不能伪造。
- 每次运行至少记录 method、seed、scenario、task_id、AA/AIA/forgetting、per-class accuracy、confusion matrix、logit gap、时间和显存。

常见误区

- 误区：跑通一次就算复现。复现还要和论文 AIA 偏差不超过预设阈值，并能解释差异。
- 误区：测试集选最佳 epoch。固定 epoch 或验证集规则必须在训练前写清楚。

分层练习：从一张主表反查：表格数字 -> tidy CSV 行 -> raw JSON -> 配置 -> 代码 commit。

本课专属任务：本课专属任务：列出 raw JSON 必须包含的字段，并说明缺 seed 或 task_id 会破坏哪些统计。

放回本项目：图 4/5 要固定方法颜色、线型和图例顺序；曲线终点必须与表 2 final AA 完全一致。

练习与自检：一条方法曲线 AIA 高但 final AA 低，可能说明什么？

参考答案：它早期阶段表现较好，但随增量推进下降较快；需要看遗忘与后期稳定性。

第 36 课 稳定性-可塑性散点图

学完这一课，你应该会

- 把 old/new 映射到二维
- 解释右上角
- 避免单点过度解读

读图

横轴 old accuracy 表示稳定性，纵轴 new accuracy 表示可塑性。右上角意味着旧类保得住、新类也学得好；左上是偏新类，右下可能过度稳定、学不动新类。

成本编码

点大小可编码训练时间或额外参数，但若造成遮挡就应分开画性能-成本图。散点应带误差条或按种子显示，避免均值点掩盖波动。

零基础加厚讲解

先修回顾：工程部分的目标是可复现。你要让未来任何人能从配置、日志、脚本重新生成同一张表。

直觉解释

raw 日志是事实源，tidy CSV 是统计入口，tables/figures 是展示结果。展示层不能手改数字，否则证据链断掉。

可复现不只是固定 seed，还包括固定代码 commit、依赖版本、数据处理、checkpoint 选择规则、输出目录和校验脚本。

放到项目里怎么用

- 当前官方 CLearning 固定在 commit ce0789a40...48f0；Python 3.12 下 continuum==1.2.4 安装阻塞，所以正式 B50-5S 结果不能伪造。
- 每次运行至少记录 method、seed、scenario、task_id、AA/AIA/forgetting、per-class accuracy、confusion matrix、logit gap、时间和显存。

常见误区

- 误区：跑通一次就算复现。复现还要和论文 AIA 偏差不超过预设阈值，并能解释差异。
- 误区：测试集选最佳 epoch。固定 epoch 或验证集规则必须在训练前写清楚。

分层练习：从一张主表反查：表格数字 -> tidy CSV 行 -> raw JSON -> 配置 -> 代码 commit。

本课专属任务：本课专属任务：解释固定 commit、固定配置和固定 epoch 为什么比口头说“我按论文跑的”更可信。

放回本项目：D0 位于近似左上极端：old≈0、new≈100；D3/D4 向右移动，说明 replay 恢复稳定性，同时 new 仍较高。

练习与自检：如果一个方法 old=80、new=20，应该怎样描述？

参考答案：稳定性较强但可塑性不足，可能约束过强或新类训练不足；不能只因 old 高就称优。

第 37 课 混淆矩阵

学完这一课，你应该会

- 读懂行与列
- 识别最新类吸附区域
- 使用归一化矩阵

结构

通常行是真实类别、列是预测类别；每行归一化后，对角线是该类正确率。若大量旧类行在最新类列出现高值，说明旧样本被吸附到最新类。绘图必须注明方向，避免行列误读。

分阶段分组

100 类矩阵较密，可按增量阶段画分隔线或只标关键区域。不要只用花哨热图，需配合具体比例和结论。

零基础加厚讲解

先修回顾：工程部分的目标是可复现。你要让未来任何人能从配置、日志、脚本重新生成同一张表。

直觉解释

raw 日志是事实源，tidy CSV 是统计入口，tables/figures 是展示结果。展示层不能手改数字，否则证据链断掉。

可复现不只是固定 seed，还包括固定代码 commit、依赖版本、数据处理、checkpoint 选择规则、输出目录和校验脚本。

放到项目里怎么用

- 当前官方 CLearning 固定在 commit ce0789a40...48f0；Python 3.12 下 continuum==1.2.4 安装阻塞，所以正式 B50-5S 结果不能伪造。
- 每次运行至少记录 method、seed、scenario、task_id、AA/AIA/forgetting、per-class accuracy、confusion matrix、logit gap、时间和显存。

常见误区

- 误区：跑通一次就算复现。复现还要和论文 AIA 偏差不超过预设阈值，并能解释差异。
- 误区：测试集选最佳 epoch。固定 epoch 或验证集规则必须在训练前写清楚。

分层练习：从一张主表反查：表格数字 -> tidy CSV 行 -> raw JSON -> 配置 -> 代码 commit。

本课专属任务：本课专属任务：针对 continuum 安装阻塞写一段状态说明：阻塞是什么、影响什么、不影响什么。

放回本项目：诊断混淆矩阵用于确认 bus 错误是否流向 train；正式图 7 只为关键方法生成并按任务阶段分组。

练习与自检：混淆矩阵对角线低能否直接说明遗忘？

参考答案：不一定，也可能该类从未学好。需与历史阶段准确率比较才能称为遗忘。

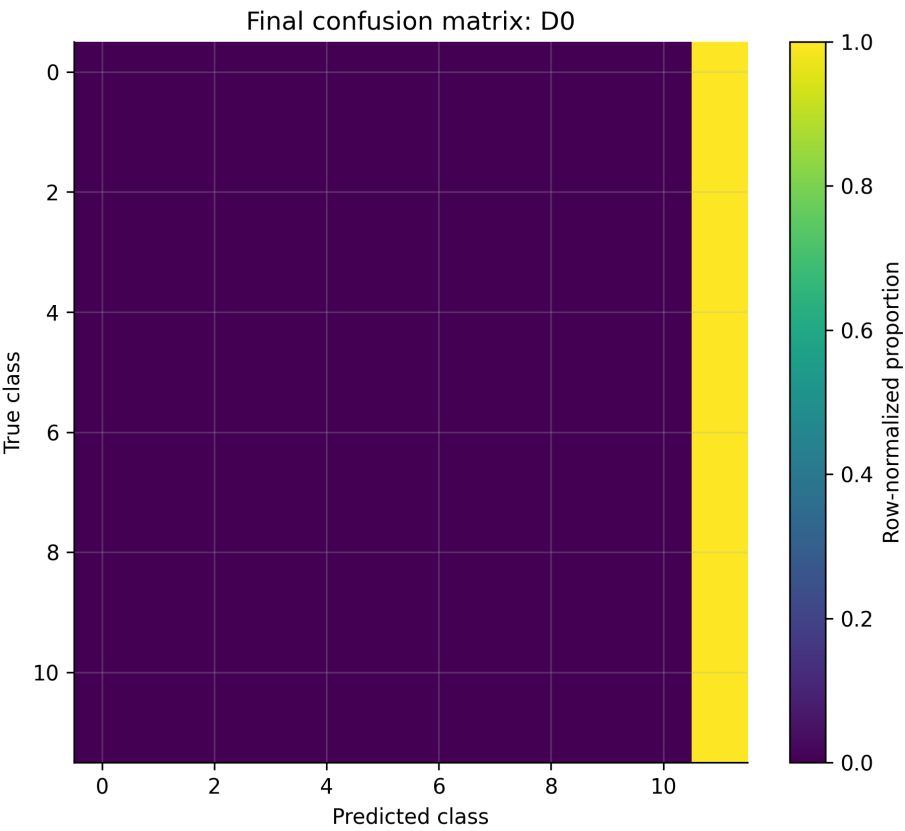


图 4 本项目诊断混淆矩阵示例；阅读前确认行/列定义。

第 38 课 性能-成本图

学完这一课，你应该会

- 同时衡量准确率与计算代价
- 理解参数、显存、时间
- 读懂 AIA/训练小时

成本维度

多教师会增加教师前向与显存，即使最终学生参数不增加，训练成本也可能翻倍。应记录模型参数量、额外教师参数、峰值显存、单阶段/总训练时间。

效率指标

AIA/训练小时是一种直观效率，但不是唯一标准；训练时间受硬件和实现影响，必须在同一设备同一环境测量。性能-成本散点的理想区域是左上：成本低、AIA 高。

零基础加厚讲解

先修回顾：工程部分的目标是可复现。你要让未来任何人能从配置、日志、脚本重新生成同一张表。

直觉解释

raw 日志是事实源，tidy CSV 是统计入口，tables/figures 是展示结果。展示层不能手改数字，否则证据链断掉。

可复现不只是固定 seed，还包括固定代码 commit、依赖版本、数据处理、checkpoint 选择规则、输出目录和校验脚本。

放到项目里怎么用

- 当前官方 CLearning 固定在 commit ce0789a40...48f0；Python 3.12 下 continuum==1.2.4 安装阻塞，所以正式 B50-5S 结果不能伪造。
- 每次运行至少记录 method、seed、scenario、task_id、AA/AIA/forgetting、per-class accuracy、confusion matrix、logit gap、时间和显存。

常见误区

- 误区：跑通一次就算复现。复现还要和论文 AIA 偏差不超过预设阈值，并能解释差异。
- 误区：测试集选最佳 epoch。固定 epoch 或验证集规则必须在训练前写清楚。

分层练习：从一张主表反查：表格数字 -> tidy CSV 行 -> raw JSON -> 配置 -> 代码 commit。

本课专属任务：本课专属任务：设计一个 sanity check：图中 final AA 必须等于主表 final AA，否则拒绝发布。

放回本项目：表 5与图 8用于回答“MTD 的提升是否值得额外计算”。若提升小于随机波动而成本明显增加，应停止该方向。

练习与自检：学生最终参数相同，能否说多教师没有额外成本？

参考答案：不能。训练时仍需多个教师前向、存储和显存；必须报告训练成本。

第九部分 读论文与判断研究价值

第 39 课 怎样读 LwF 论文

学完这一课，你应该会

- 按问题-方法-协议-结论阅读
- 区分原论文设定与当前 CIL
- 复现核心损失

阅读顺序

先看论文解决什么限制：无旧数据时保留旧能力。再看训练输入和输出头、损失公式、温度与权重。随后看数据集、任务顺序、是否给任务 ID、评价指标。最后才看结果表。

迁移到本项目

原论文的任务设置不必等同于当前单头 CIFAR-100 CIL。复现时应把“论文思想”和“本地协议适配”分开写，避免把不相同的数字当作失败或成功。

零基础加厚讲解

先修回顾：读论文的目标不是背摘要，而是把论文问题、方法假设、实验协议、可复现代码和本项目边界对应起来。

直觉解释

LwF、PODNet、MTD、SS-IL 都有自己的原始问题设定。把它们迁移到当前项目时，要检查数据集、backbone、memory、任务划分、评价指标是否一致。

综述适合建立地图，原论文适合查公式和协议，官方代码适合核对实现细节，本地日志适合支持你自己的结论。四者不能互相替代。

放到项目里怎么用

- MTD 论文用于理解多教师构造；CLearning 官方配置用于复现；交接文档用于旧故障事实；本地 raw 日志用于最终表图。
- 下一步研究方向由正式结果触发，不由个人偏好触发。若多教师无稳定收益，就停止该方向。

常见误区

- 误区：论文说有效，本地就一定有效。协议、代码和环境变化都可能造成差异。
- 误区：官方已有的组合方法还能包装成新方法。已有配置只能作为基线。

分层练习：读任何一篇方法论文时，先抄四行：解决什么问题、核心损失是什么、实验协议是什么、与本项目哪里不同。

本课专属任务：本课专属任务：读 LwF 时分别记录原论文任务设定和本项目 CIL 设定，指出不可直接比较的地方。

放回本项目：本地论文 00_Learning_without_Forgetting.pdf 是理论学习来源；项目 LwF 是 sanity baseline，不承担主创新。

练习与自检：读方法论文时为什么不能先只抄最好数字？

参考答案：因为数字依赖任务设定、数据、骨干和评估协议；不理解协议就无法公平比较。

第 40 课 怎样读 MTD 论文

学完这一课，你应该会

- 找到教师构造机制
- 理解消融表
- 验证官方实现与论文一致

重点问题

教师从哪里来？如何产生多样性？每个教师覆盖哪些类？蒸馏哪些层/输出？教师损失如何聚合？新增成本多少？这些问题比“用了两个教师”更关键。

看消融

消融应分别比较单教师、多教师，不同多样性策略和教师数量；还要看不同基础方法上的稳定性。若只在一个设置改善，可能是偶然或特定超参数。

零基础加厚讲解

先修回顾：读论文的目标不是背摘要，而是把论文问题、方法假设、实验协议、可复现代码和本项目边界对应起来。

直觉解释

LwF、PODNet、MTD、SS-IL 都有自己的原始问题设定。把它们迁移到当前项目时，要检查数据集、backbone、memory、任务划分、评价指标是否一致。

综述适合建立地图，原论文适合查公式和协议，官方代码适合核对实现细节，本地日志适合支持你自己的结论。四者不能互相替代。

放到项目里怎么用

- MTD 论文用于理解多教师构造；CLearning 官方配置用于复现；交接文档用于旧故障事实；本地 raw 日志用于最终表图。
- 下一步研究方向由正式结果触发，不由个人偏好触发。若多教师无稳定收益，就停止该方向。

常见误区

- 误区：论文说有效，本地就一定有效。协议、代码和环境变化都可能造成差异。
- 误区：官方已有的组合方法还能包装成新方法。已有配置只能作为基线。

分层练习：读任何一篇方法论文时，先抄四行：解决什么问题、核心损失是什么、实验协议是什么、与本项目哪里不同。

本课专属任务：本课专属任务：读 MTD 时记录教师如何生成、教师如何聚合、成本如何增加，以及本项目如何复现。

放回本项目：本地 01_Class_Incremental_Learning_with_Multi-Teacher_Distillation.pdf 与官方 CLearning 固定提交共同作为实现依据；MTD-SSIL 已是官方基线，不能包装成创新。

练习与自检：为什么官方有配置仍要做代码审计？

参考答案：配置可能依赖特定 commit、默认值和环境；还需确认运行路径、教师构造、指标与论文版本一致。

第 41 课 怎样读蒸馏持续学习综述

学完这一课，你应该会

- 建立方法分类树
- 用综述找原论文
- 避免把综述当唯一证据

分类阅读

可按蒸馏位置分类：logit、feature、relation；按数据分为无重放与带重放；按场景分 Task/Domain/Class-IL；按教师分单教师、多教师或自蒸馏。先建立地图，再深入与你项目直接相关的分支。

证据层级

综述适合找术语、脉络和代表方法，但具体公式、实验协议和数字应回到原论文与官方代码核对。综述的分类也可能随年份更新。

零基础加厚讲解

先修回顾：读论文的目标不是背摘要，而是把论文问题、方法假设、实验协议、可复现代码和本项目边界对应起来。

直觉解释

LwF、PODNet、MTD、SS-IL 都有自己的原始问题设定。把它们迁移到当前项目时，要检查数据集、backbone、memory、任务划分、评价指标是否一致。

综述适合建立地图，原论文适合查公式和协议，官方代码适合核对实现细节，本地日志适合支持你自己的结论。四者不能互相替代。

放到项目里怎么用

- MTD 论文用于理解多教师构造；CLearning 官方配置用于复现；交接文档用于旧故障事实；本地 raw 日志用于最终表图。
- 下一步研究方向由正式结果触发，不由个人偏好触发。若多教师无稳定收益，就停止该方向。

常见误区

- 误区：论文说有效，本地就一定有效。协议、代码和环境变化都可能造成差异。
- 误区：官方已有的组合方法还能包装成新方法。已有配置只能作为基线。

分层练习：读任何一篇方法论文时，先抄四行：解决什么问题、核心损失是什么、实验协议是什么、与本项目哪里不同。

本课专属任务：本课专属任务：读综述时只提取方法地图，不直接引用综述中的排名作为本项目结论。

放回本项目：02_Continual_Learning_With_Knowledge_Distillation_A_Survey.pdf 用来建立知识地图，并帮助解释 LwF、PODNet、MTD 的关系。

练习与自检：综述说某方法最好，能否直接作为你实验选择的证据？

参考答案：只能作为线索；需检查原论文协议与当前任务是否匹配，并由本地复现验证。

第 42 课 创新边界与下一步决策

学完这一课，你应该会

- 区分复现、组合和创新
- 使用预先定义的停止规则
- 从结果选择研究分支

什么不算创新

直接运行官方已有配置、把 MTD 与 SS-IL 组合但官方已有 MTD-SSIL、把历史 checkpoint 称为多样化教师，都不能作为新贡献。复现价值在于建立可信基线，不应伪装成算法创新。

决策树

若 MTD-PODNet 和 MTD-SSIL 都无稳定增益，停止多教师；若前者有效而后者增益消失，研究多教师收益与分类偏差的关系；若两者都稳定有效，再研究教师质量评估和自适应聚合。预先写规则可减少看到结果后讲故事。

零基础加厚讲解

先修回顾：读论文的目标不是背摘要，而是把论文问题、方法假设、实验协议、可复现代码和本项目边界对应起来。

直觉解释

LwF、PODNet、MTD、SS-IL 都有自己的原始问题设定。把它们迁移到当前项目时，要检查数据集、backbone、memory、任务划分、评价指标是否一致。

综述适合建立地图，原论文适合查公式和协议，官方代码适合核对实现细节，本地日志适合支持你自己的结论。四者不能互相替代。

放到项目里怎么用

- MTD 论文用于理解多教师构造；CLearning 官方配置用于复现；交接文档用于旧故障事实；本地 raw 日志用于最终表图。
- 下一步研究方向由正式结果触发，不由个人偏好触发。若多教师无稳定收益，就停止该方向。

常见误区

- 误区：论文说有效，本地就一定有效。协议、代码和环境变化都可能造成差异。
- 误区：官方已有的组合方法还能包装成新方法。已有配置只能作为基线。

分层练习：读任何一篇方法论文时，先抄四行：解决什么问题、核心损失是什么、实验协议是什么、与本项目哪里不同。

本课专属任务：本课专属任务：按三条停止规则判断下一阶段：双 MTD 无效、仅 PODNet 上有效、两者都有效。

放回本项目：本轮交付的科学价值是把旧故障机制讲清并建立标准基线；创新问题留到基线通过复现验收之后。

练习与自检：为什么预先写停止规则有助于科研可信度？

参考答案：它减少事后挑选有利解释和无限追加实验，让方向选择由事先标准驱动。

第十部分 组会叙事与独立答辩

第 43 课 把结果讲成一条可信故事

学完这一课，你应该会

- 按问题-证据-结论组织汇报
- 控制每页信息
- 明确已完成与待完成

推荐叙事

第一，旧结果为什么不可信：多项改动混杂、分类器继承、测试泄漏和双教师缺陷。第二，类别替换的根因：CE、KD 平移盲区、logit gap 与混淆证据。第三，D0-D4 中 replay 缓解了什么。第四，标准协议和方法矩阵。第五，正式结果将决定哪条研究分支。

每页一句话

每张表/图都先写“这页回答什么问题”，再写一句结论和一条限制。例如：D3 显著恢复旧类准确率，但 6 epoch 单种子诊断不能证明 SS-IL 优于 replay。

零基础加厚讲解

先修回顾：最后部分训练答辩能力。答辩不是背稿，而是在追问下仍能回到证据、公式、表图和边界。

直觉解释

组会叙事应按因果链组织：旧结果为何不可信 -> 类别替换机制 -> 诊断实验怎么支持 -> 标准复现如何设计 -> 结果将决定什么研究分支。

每张表和图都要回答一个问题。表 1 回答故障机制，表 2 回答主方法性能，表 4 回答多教师与偏差校正的二因素关系，图 2 回答 logit gap 是否解释准确率突变。

放到项目里怎么用

- 当前能完整讲 D0-D4 和自动表图框架；正式 B50-5S 主结果仍因依赖阻塞待运行。这个边界要主动说清楚。
- 答辩中遇到“为什么不继续做新算法”，回答是：基线尚未通过复现验收前，新算法收益无法被可信归因。

常见误区

- 误区：为了显得进展多而补数字。科研汇报宁可承认阻塞，也不能捏造未运行结果。
- 误区：只讲结论不讲限制。导师追问时，限制说得清楚反而提高可信度。

分层练习：准备 5 分钟口述：先讲一个故障数字，再讲一个公式原因，再讲一个对照实验，再讲下一步边界。

本课专属任务：本课专属任务：给每张组会图写一句“这张图回答的问题”，不要只写图名。

放回本项目：当前可完整汇报故障复盘；标准主结果只能汇报配置与阻塞状态，不能把待运行表说成已完成。

练习与自检：组会第一页结果页应该先放所有方法大表，还是先解释旧故障？

参考答案：本项目更适合先解释为何重启和故障机制，再进入标准协议，否则导师会质疑数字基础。

第 44 课 导师常问什么，怎样回答

学完这一课，你应该会

- 用证据而非口号回答
- 承认边界但给出下一步
- 处理复现偏差和负结果

五个高频问题

为什么旧类归零？为什么只看最终准确率不够？为什么做三种子？MTD 提升是否超过随机波动？提升是否值得额外成本？回答时都要指向具体公式、表或图。

回答模板

先给结论，再给证据，最后给限制与下一步。例如：“当前诊断支持分类偏差是主因，因为 D0 gap=26.99 且 old=0，加入 replay 后 old=58.73；但这是单种子短实验，正式结论要等 B50-5S 三种子。”

零基础加厚讲解

先修回顾：最后部分训练答辩能力。答辩不是背稿，而是在追问下仍能回到证据、公式、表图和边界。

直觉解释

组会叙事应按因果链组织：旧结果为何不可信 -> 类别替换机制 -> 诊断实验怎么支持 -> 标准复现如何设计 -> 结果将决定什么研究分支。

每张表和图都要回答一个问题。表 1 回答故障机制，表 2 回答主方法性能，表 4 回答多教师与偏差校正的二因素关系，图 2 回答 logit gap 是否解释准确率突变。

放到项目里怎么用

- 当前能完整讲 D0-D4 和自动表图框架；正式 B50-5S 主结果仍因依赖阻塞待运行。这个边界要主动说清楚。
- 答辩中遇到“为什么不继续做新算法”，回答是：基线尚未通过复现验收前，新算法收益无法被可信归因。

常见误区

- 误区：为了显得进展多而补数字。科研汇报宁可承认阻塞，也不能捏造未运行结果。
- 误区：只讲结论不讲限制。导师追问时，限制说得清楚反而提高可信度。

分层练习：准备 5 分钟口述：先讲一个故障数字，再讲一个公式原因，再讲一个对照实验，再讲下一步边界。

本课专属任务：本课专属任务：准备导师追问“为什么旧结果不可信”的 60 秒答案，必须包含测试泄漏、权重继承和 KD 盲区。

放回本项目：组会答辩清单.md 可用于口头演练；本讲义附录也给出问题框架。

练习与自检：导师说“D4 只比 D3 高 0.18，做 SS-IL 有意义吗？”怎样答？

参考答案：当前没有证据说有稳定增益，短诊断中二者近似；SS-IL 的价值需在完整三种子标准协议中检验，若增益不超过波动就不宣称有效。

第 45 课 最终实操考核：从零到可答辩

学完这一课，你应该会

- 能独立画出整个系统
- 能运行并核验一次实验
- 能用表图回答研究问题

考核任务

一，口头解释神经网络训练、CE、KD 共同平移盲区。二，画出 B50-5S 六阶段。三，从配置启动一个 sanity run，检查 raw JSON。四，从 raw 生成 tidy、三线表和曲线，手算一个 AIA 核对。五，用 D0-D4 表和图讲 5 分钟故障机制。

通过标准

不靠背稿能解释每个指标；知道结果由哪个文件产生；能指出哪些结论是事实、哪些是待验证；遇到异常先查数据、形状、配置、日志和梯度，而不是盲目换超参数。

零基础加厚讲解

先修回顾：最后部分训练答辩能力。答辩不是背稿，而是在追问下仍能回到证据、公式、表图和边界。

直觉解释

组会叙事应按因果链组织：旧结果为何不可信 -> 类别替换机制 -> 诊断实验怎么支持 -> 标准复现如何设计 -> 结果将决定什么研究分支。

每张表和图都要回答一个问题。表 1 回答故障机制，表 2 回答主方法性能，表 4 回答多教师与偏差校正的二因素关系，图 2 回答 logit gap 是否解释准确率突变。

放到项目里怎么用

- 当前能完整讲 D0-D4 和自动表图框架；正式 B50-5S 主结果仍因依赖阻塞待运行。这个边界要主动说清楚。
- 答辩中遇到“为什么不继续做新算法”，回答是：基线尚未通过复现验收前，新算法收益无法被可信归因。

常见误区

- 误区：为了显得进展多而补数字。科研汇报宁可承认阻塞，也不能捏造未运行结果。
- 误区：只讲结论不讲限制。导师追问时，限制说得清楚反而提高可信度。

分层练习：准备 5 分钟口述：先讲一个故障数字，再讲一个公式原因，再讲一个对照实验，再讲下一步边界。

本课专属任务：本课专属任务：完成闭卷验收：画协议、讲公式、找日志、核表图、说边界，五项都通过才算接住项目。

放回本项目：真正的“学会”不是看完 PDF，而是能在项目目录中找到证据、复跑、核算并回答质疑。建议按第 1-21 课学习概念，再用第 22-45 课完成一次闭环。

练习与自检：你什么时候可以说自己接住了这个项目？

参考答案：当你能独立解释故障机制、复现标准基线、验证日志到表图的一致性，并诚实界定创新与未完成部分时。

深度工作纸 把 45 课讲厚、讲透、讲到能答辩

这一部分不是新目录，而是把前面每课的关键知识展开成可手算、可复述、可落到项目文件的长讲。使用方法：先读对应课程，再做这里的工作纸；每张工作纸都要求你能闭卷说出结论、证据和边界。

工作纸 1：从一张图片到一个 logit

你要真正理解的流程

一张 CIFAR-100 图片不是“图片”这个抽象词，而是 $3 \times 32 \times 32$ 个数字。三个通道分别表示红、绿、蓝，每个位置是一个像素强度。网络第一层看到的的就是这些数字，不知道 bus、train、wolf 这些人类词汇。

卷积层把局部像素组合成局部特征，例如边缘、颜色块、纹理。越往后，特征越抽象。最后分类器拿到一个特征向量 x ，用每个类别自己的权重行 W_c 计算 $z_c = W_c x + b_c$ 。 z_c 就是这个类别的 logit。

分类器不是在问“图片是不是 bus”，而是在给所有已学类别同时打分。单头 CIL 最终只看所有已学类别中哪个 logit 最大。因此旧类失败常常不是旧类 logit 变成负无穷，而是新类 logit 更大。

本项目手算例子

假设某张 bus 图片在 12 类模型中得到三个关键分数：bus=3.1、train=7.4、wolf=1.2。模型会预测 train。此时 bus 的分数并不低，但它输给了 train，这就叫决策层面的替换。

如果所有旧类 logits 同时下降 4，bus 从 3.1 变成 -0.9，而 train 仍是 7.4，替换会更严重。old/new logit gap 正是为了度量这个新旧分数尺度差。

你在代码里应该找什么

找模型 forward 的输出变量，通常叫 logits、outputs 或 pred。确认它的 shape 是 [batch, num_classes]。

找最后分类器模块，通常是 fc、classifier 或 linear。确认增量阶段 num_classes 是否正确扩展。

找评估函数，确认预测是 argmax over all learned classes，而不是只在当前任务类别里 argmax。

自测答案

问题：为什么不能只看 softmax 概率？答案：softmax 是相对概率，受所有类别共同影响。诊断类别替换时必须看 logits、gap、per-class accuracy 和混淆矩阵。

问题：为什么 RGB/BGR 错误会严重？答案：通道顺序改变了输入数字的语义，网络学到的颜色和纹理对应关系会错。若同时改变优化器和 epoch，就无法单独归因。

工作纸 2：CE 为什么会把新类推上去、把旧类压下去

从公式到方向

交叉熵 $CE = -\log p_y$ 。这里 y 是真实类别， p_y 是 softmax 后真实类别概率。若 p_y 很小，损失很大；若 p_y 接近 1，损失接近 0。训练的目标就是让损失下降。

对每个 logit z_j ，CE 的梯度是 $p_j - 1[j=y]$ 。当 j 是真实类时，这个数通常为负，梯度下降会让 z_y 上升；当 j 不是真实类时，这个数为正，梯度下降会让 z_j 下降。

这条公式足以解释新类单独训练的偏置。新阶段 batch 里全是 train，train 是真实类，bus 和所有旧类都是非真实类。于是 CE 一遍遍推高 train，同时压低 bus 和其他旧类。

用数字看一遍

假设当前样本真实类是 train，softmax 后 $\text{train}=0.60$ 、 $\text{bus}=0.20$ 、其他类合计 0.20。train 梯度是 -0.40，更新后 train logit 会升；bus 梯度是 +0.20，更新后 bus logit 会降。

如果训练集没有 bus 样本，下一批、下下一批仍然没有力量告诉模型 bus 应该被提高。短期看新类准确率很好，长期看旧类被系统性压低。

D0-D4 如何验证

D0 当前实现出现 old accuracy 接近 0，gap=26.99，说明新旧分数尺度已经严重偏向新类。

D3 加入每类 20 个旧样本后 old accuracy 恢复到 58.73。旧样本让 CE 在部分 batch 中也把旧类当真实类推高，因此能抵消一部分新类偏置。

D4 加 separated softmax 后 old accuracy 约 58.91，与 D3 接近。当前诊断只能说 replay 是强缓解因素，不能夸大 D4 的额外收益。

自测答案

问题：新类 CE 为什么影响旧类？答案：softmax 是多类别竞争，非真实类 logit 的 CE 梯度为正，梯度下降会降低它们。

问题：如果只训练新类但冻结旧分类器行，能彻底解决吗？答案：不一定。冻结旧行能减少分类器漂移，但 backbone 特征和新类行仍可能改变决策边界。

工作纸 3：KD 的作用、盲区和共同平移推导

KD 在做什么

知识蒸馏 KD 让学生模型的输出分布接近教师模型。教师通常是上一个阶段训练好的旧模型，学生是当前要学新类的模型。KD 的直觉是：即使没有完整旧数据，也让学生别偏离教师太远。

蒸馏常用 KL 散度比较两个 softmax 分布。temperature T 会把 logits 除以 T 后再 softmax，T 越大分布越平滑，能保留更多类别间相似性信息。

共同平移为什么是盲区

$\text{softmax}(z)_c = \exp(z_c) / \sum_j \exp(z_j)$ 。如果给所有旧类 logits 都加同一个常数 a，分子和分母都会乘 $\exp(a)$ ，这个因子抵消，所以 $\text{softmax}(z+a) = \text{softmax}(z)$ 。

这意味着：只在旧类别内部做 softmax 的 KD，只能看见旧类之间的相对关系。bus 比 car 高、car 比 road 高这些关系能看见；所有旧类一起比新类低了 10 分，它看不见。

项目诊断里 KD common shift delta 约 $1.19e-7$ ，就是数值层面证明这个不变性：对旧类 logits 共同平移，旧类内部 KD 基本不变。

它如何解释 bus -> train -> wolf

在 11->12 阶段，train 是新类。CE 推高 train 并压低旧类，旧类内部 KD 仍可能认为 bus、car、road 的相对关系没变。最后整体决策时 train logit 最大，bus 样本被判 train。

下一阶段 wolf 成为最新类，同样机制再次发生，train 又变成旧类并被 wolf 吸走。这就是连续替换，而不是 bus 或 train 本身特殊。

自测答案

问题：KD 失效是否说明 KD 没用？答案：不是。它说明当前 KD 范围和数据条件不足以约束新旧类共同竞争，需要 replay、全局校准或 separated softmax 等机制配合。

问题：为什么要记录 old/new logit gap？答案：它直接捕捉旧类整体下移和新类整体上移，而这是旧类内部 KD 看不见的。

工作纸 4：CIL 协议、B50-5S 和指标手算

协议先于方法

任何持续学习结果都必须先说明协议。训练时每阶段看到哪些类？是否允许旧样本？测试时给不给任务 ID？使用单头还是多头？如果这些不清楚，方法名字没有意义。

本项目正式协议是 CIFAR-100、ResNet-32、B50-5S、每类 20 exemplar、三种子。B50-5S 表示首任务 50 类，后续 5 个阶段，每阶段增加 10 类。共有 6 个评价点。

AA、AIA、forgetting

AA 是某阶段在所有已学类别上的平均准确率。若 Task0 到 Task5 的 AA 分别是 70、65、60、58、55、52，则 $AIA = (70 + 65 + 60 + 58 + 55 + 52) / 6 = 60$ 。

forgetting 衡量旧任务从历史最好到当前的下降。若 bus 所属任务历史最好 80，最后只有 30，则该任务遗忘为 50。通常需要对旧任务取平均。

old accuracy 和 new accuracy 是诊断稳定性-可塑性的关键。old 高说明保旧好，new 高说明学新好；两者必须一起看。

为什么三种子

神经网络训练有随机初始化、数据顺序、数据增强、CUDA 算法等随机来源。单个 seed 可能碰巧好或坏，所以正式结论至少需要均值和标准差。

若 MTD-PODNet 比 PODNet 平均高 0.4，但标准差是 0.8，就不能说稳定有效。正确表述是：当前未观察到超过随机波动的稳定增益。

自测答案

问题：为什么不能把旧 VGG D0-D4 和 ResNet-32 B50-5S 放进同一排名？答案：模型、协议、阶段、目的都不同，一个是故障诊断，一个是正式复现。

问题：AIA 是最终准确率吗？答案：不是。AIA 是多个阶段 AA 的平均，反映整个增量过程。

工作纸 5：方法矩阵一口气讲清

先按信息来源分类

FineTune 只用新阶段数据训练，通常作为遗忘下界。LwF 不保存旧样本，靠旧模型输出蒸馏。Replay 保存少量旧样本，让训练重新看到旧类。SS-IL 处理新旧类 softmax 竞争偏差。PODNet 蒸馏中间特征，保护表示。MTD 用多个教师提供更丰富蒸馏信号。

这些方法不是同一层级。FineTune/LwF/Replay 是 sanity；PODNet 和 MTD-PODNet 是主蒸馏基线；SS-IL 和 MTD-SSIL 是偏差校正相关基线。

二因素设计

表 4 的 2x2 很关键：普通分类下比较 PODNet 与 MTD-PODNet，偏差校正下比较 SS-IL 与 MTD-SSIL。横向看多教师增益，纵向看偏差校正增益，最后看交互增益。

如果 MTD-PODNet 有提升但 MTD-SSIL 没提升，可能说明多教师收益被偏差校正吸收，下一阶段应研究多教师和分类偏差的关系。

MTD 的边界

真正 MTD 不是随便拿两个 checkpoint。教师必须覆盖相同或可比较的知识空间，并通过论文定义的扰动或构造方式形成有意义多样性。

10 类教师没有 bus 之后的知识，11 类教师有更多类别，把它们混在一起保护 12 类学生，会出现覆盖不一致。再加上无学生梯度损失项，旧双教师实现不能作为正式 MTD 结果。

自测答案

问题：MTD-SSIL 能不能当创新？答案：不能，官方配置已经存在，只能作为基线。

问题：为什么 Replay 还要做？答案：它是最直接检验样本不平衡和新类偏置的 sanity baseline。

工作纸 6：旧 VGG 故障审计清单

审计顺序

第一看数据。确认 CIFAR-100 是否原始读取，RGB/BGR 是否正确，是否被转 JPEG，是否放大到 224x224。数据格式错误会污染所有后续结论。

第二看分类器。增量扩类时是否复制旧权重和偏置？新类行如何初始化？optimizer 是否包含旧行？预热阶段是否更新了整个分类层？

第三看训练数据和损失。当前阶段是否只有新类？CE 和 KD 的权重是多少？KD softmax 覆盖哪些类别？教师输出是否 detach，学生输出是否保留梯度？

第四看 checkpoint 选择。是否用测试准确率选择最佳 epoch？如果是，正式评价存在测试泄漏。

把问题变成实验

不要只说“可能是分类器初始化问题”。要设计 D1：正确复制旧权重但仍更新整个分类层。若 D1 仍替换，说明初始化不是唯一因素。

不要只说“可能是样本不平衡”。要设计 D3：当前 KD 加每类 20 个旧样本 replay。若 old accuracy 大幅恢复，说明旧样本信号是强因素。

汇报边界

旧 VGG 审计结论可以说：旧结果不适合作为正式对比，因为存在数据、初始化、训练、KD、测试泄漏等混杂因素。

但不能说：我们已经证明所有 CIL 方法都会发生 bus->train->wolf。D0-D4 是故障机制诊断，不是跨方法结论。

自测答案

问题：为什么 10->11 重初始化很严重？答案：旧类分类器模板被直接丢掉，模型最后一层不再知道旧类该如何打分。

问题：为什么测试集选 epoch 会泄漏？答案：测试结果参与了模型选择，最终测试不再是独立评价。

工作纸 7：D0-D4 数字如何讲成证据链

先记住关键数字

D0 当前实现：bus 0.00、train 100.00、old 0.00、gap 26.99。它复现了极端替换，说明故障不是口头猜测。

D1 正确复制旧权重但仍更新整个分类层：bus 0.00、train 100.00、old 0.18、gap 19.37。初始化修复后仍几乎失败，说明还有训练偏置或漂移。

D2 冻结旧分类器只训练新类行：bus 0.00、train 100.00、old 1.36、gap 21.60。仅冻结旧行仍不足以恢复旧类，说明新类行和特征/尺度仍可支配。

D3 replay：bus 10.00、train 94.00、old 58.73、gap 13.53。旧类样本显著恢复 old accuracy，说明样本不平衡和新类 CE 是关键。

D4 replay + separated softmax：bus 10.00、train 94.00、old 58.91、gap 13.78。与 D3 接近，当前不能夸大 D4。

讲法模板

第一句给结论：旧实现的类别替换主要表现为新类 logit 尺度压过旧类，单靠旧类内部 KD 无法阻止。

第二句给证据：D0 old=0 且 gap=26.99；给旧类 replay 后 old 恢复到 58.73，说明旧样本信号强烈缓解替换。

第三句给边界：D0-D4 是固定种子、短 epoch 诊断，不能作为正式方法排名。

图 1 和图 2 怎么配合

图 1 展示 bus/train/wolf 准确率轨迹，回答“替换现象是否发生”。图 2 展示 logit gap，回答“替换是否与决策尺度漂移同步”。

若某个 epoch bus 准确率突然归零，同时 gap 突然跨过阈值，这比单独看准确率更有解释力。

自测答案

问题：D3 old 提升能不能证明 Replay 是最终最好方法？答案：不能。它只证明在旧故障诊断中加入旧样本缓解极端替换。

问题：为什么 bus 仍只有 10？答案：replay 缓解整体 old，但单个类仍可能难，需看 per-class 和混淆矩阵。

工作纸 8：从 raw 日志到三线表

证据链

raw JSON 是不可修改事实源。每次实验运行都写 method、seed、scenario、task_id、learned_classes、overall、old、new、AA、AIA、forgetting、per-class accuracy、confusion matrix、logit gap、显存和时间。

tidy CSV 是统计层，一行通常对应一个 method-seed-task。所有均值、标准差、AIA、forgetting 都从 tidy 计算。最终表格只读取 tidy，不允许手填。

表格和图片是展示层。展示层可以改标题、字体、排序，但不能改数字。若发现数字不对，应回到计算函数或 raw，而不是直接改 Word/PPT 里的数。

三线表规范为什么重要

三线表不是形式主义。没有竖线和内部网格能迫使你设计清楚；均值±标准差能迫使你面对随机性；表注能迫使你说明协议。

表 2 是主结果表，回答哪个方法在标准协议下综合表现最好。表 3 是逐阶段表，回答方法是早期领先还是后期抗遗忘。表 5 是效率表，回答收益是否值得成本。

校验规则

表中 AIA 必须等于逐阶段 AA 算术平均。图中曲线终点必须等于主结果表 final AA。缺 seed 时拒绝生成 mean±std。

不同协议不能放进同一排名。D0-D4、旧 VGG、B50-5S ResNet-32 必须分开。

自测答案

问题：为什么表下注明 memory 和 seeds？答案：memory 改变方法信息量，seeds 决定稳定性统计，两者都会影响可比性。

问题：为什么不手抄数字？答案：手抄无法追溯，容易错，并破坏复现实验的证据链。

工作纸 9：八类图分别回答什么问题

诊断图

图 1 类别替换轨迹回答：bus、train、wolf 是否出现接力式替换。横轴是 epoch 或阶段，纵轴是 per-class accuracy。

图 2 logit gap 轨迹回答：准确率突变是否来自新旧 logit 尺度漂移。它应与图 1 对齐阅读。

图 3 CE/KD 梯度方向回答：CE 和 KD 分别怎样作用于旧类行和新类行。它是公式解释与代码行为之间的桥。

正式结果图

图 4 增量准确率曲线回答：不同方法在各阶段 AA 如何变化，是否只是起点高，还是后期抗遗忘。

图 5 遗忘曲线回答：旧任务性能下降多少。它能区分新类学得好和旧类忘得少。

图 6 稳定性-可塑性散点图回答：old accuracy 和 new accuracy 是否兼顾。右上角才是理想区域。

图 7 混淆矩阵回答：错误集中在哪里，旧类是否被最新类别吸走。

图 8 性能-成本图回答：多教师提升是否值得额外训练时间、显存或参数。

图表写作规范

每张图必须有坐标轴、单位、图例和一句结论。三种子曲线用均值实线和标准差阴影。颜色和线型在所有图中保持一致。

不要用 3D、渐变背景或装饰图标。科研图的任务是减少误解，不是增加视觉噪声。

自测答案

问题：为什么混淆矩阵要按增量阶段分组？答案：这样能看出旧阶段类别是否系统性流向新阶段类别。

问题：为什么稳定性-可塑性图有用？答案：它把 old/new 分开，避免 overall 掩盖偏新或偏旧。

工作纸 10：论文阅读到项目决策

读论文的四张卡片

第一张卡片写问题：论文解决的是无旧数据、少旧样本、小任务、大任务，还是分类偏差？第二张写方法：核心损失和训练信号是什么？第三张写协议：数据集、backbone、memory、任务划分、seed。第四张写和本项目差异。

LwF 的关键是不用旧数据，通过旧模型输出保护旧知识。PODNet 的关键是特征蒸馏。MTD 的关键是多教师蒸馏。SS-IL 的关键是 separated softmax 处理新旧类偏差。

从结果到下一步

如果 MTD-PODNet 和 MTD-SSIL 都没有稳定增益，停止多教师方向。此时继续调多教师容易变成追数字。

如果 MTD-PODNet 有效但 MTD-SSIL 增益消失，研究多教师收益是否主要来自缓解分类偏差，或者被 SS-IL 已经吸收。

如果两种 MTD 都稳定有效，才进入教师质量评估和自适应聚合。这个顺序能避免在基线不稳时做复杂创新。

证据边界

论文结果是外部证据，本地复现是项目证据。两者不一致时，先查协议、代码 commit、依赖、数据处理和指标计算，而不是立刻宣布论文错或方法无效。

当前正式 B50-5S 多方法三种子尚未完成，因此所有主结果表的数字都应留空或标为待实验。已经完成的是 D0-D4 诊断和表图自动化框架。

自测答案

问题：为什么不能把 MTD-SSIL 包装成创新？答案：官方已有该配置，项目中只能作为多教师偏差基线。

问题：什么时候可以提出新算法？答案：标准复现通过、基线稳定、现象清晰，并且新想法解决了已定位的具体缺口之后。

工作纸 11：组会答辩 10 分钟脚本

第 1-2 分钟：为什么重启

开场不要直接讲复杂方法。先说旧实验不适合作为正式结果：RGB/BGR 修复、优化器切换、epoch 增加同时发生；10->11 分类器重初始化；11->12 预热更新整个分类层；测试集用于选最佳 epoch；双教师损失存在实现缺陷。

结论是：旧结果保留为故障案例，不进入正式主结果排名。

第 3-5 分钟：故障机制

用 CE 梯度解释新类 CE 为什么推高新类、压低旧类。用 softmax 共同平移不变性解释旧类内部 KD 为什么看不到旧类整体下移。

给 D0-D4 数字：D0 old=0、gap=26.99；D3 replay 后 old=58.73。说明极端替换与 logit gap 和旧样本缺失强相关。

第 6-8 分钟：标准计划

说明正式协议：CIFAR-100、ResNet-32、B50-5S、20 exemplar/class、seeds 1993/1994/1995。方法矩阵包括 FineTune、LwF、Replay、PODNet、MTD-PODNet、SS-IL、MTD-SSIL。

强调所有表图由 raw 日志自动生成，不手抄数字。主表报告 AIA、final AA、forgetting、old/new accuracy 和成本。

第 9-10 分钟：边界和下一步

说清楚当前边界：正式 B50-5S 结果因 continuum/Python 依赖阻塞尚未完成，不补造数字。下一步先打通官方环境，复现 PODNet/MTD-PODNet 到论文 2 个百分点内，再扩展三种子。

最后给决策规则：若多教师无稳定收益，停止；若只在 PODNet 有效，研究多教师与偏差校正关系；若两者都有效，再做教师质量评估。

自测答案

导师问：为什么旧实验准确率突然归零？答：新类 CE 推高最新类并压低旧类，旧类内部 KD 看不到共同下移，分类器继承/更新问题放大了这个偏差，D0-D4 和 logit gap 支持该解释。

导师问：现在有没有正式结论？答：有故障机制诊断结论；标准方法优劣结论还没有，因为正式三种子复现尚未完成。

工作纸 12：零基础学习路线和验收方式

第一遍怎么学

第 1-10 课只做一件事：把神经网络训练闭环讲顺。你要能画出 图片 -> 张量 -> 特征 -> logits -> softmax -> loss -> gradient -> update。

第 11-21 课建立 CIL 方法地图。每学一个方法，都写三句话：它解决什么问题、用什么额外信号、有什么盲区。

第 22-28 课专攻交接故障。这里要背关键数字，但不是死背，而是用数字支撑因果链。

第二遍怎么练

第 29-38 课练指标和工程。你要亲手从 raw 或示例数字算一次 AIA、forgetting、mean $\pm$ std，并检查图表终点一致。

第 39-45 课练论文和答辩。每读一篇论文，只保留和项目决策有关的信息。每张组会图都写一句它回答的问题。

通过标准

口述标准：不看讲义，三分钟讲 CE/KD 为什么导致类别替换。操作标准：能在项目目录中找到 raw、tidy、tables、figures 的对应关系。判断标准：能说出哪些结论已验证、哪些仍待正式实验。

如果你只能背方法名，还不算接住项目。如果你能指出一个表格数字来自哪个日志、一个图回答哪个问题、一个方法提升是否超过标准差，就开始具备答辩能力。

自测答案

问题：遇到看不懂的公式怎么办？答案：先问它连接项目里的哪个变量，是 logits、probability、loss、gradient 还是 metric。再用一个两类或三类数字例子手算。

问题：遇到导师追问怎么办？答案：先给结论，再给证据，再给边界。不要为了显得完整而编未运行结果。

附录 A 核心术语速查

术语	项目中的含义
logit	Softmax 前的类别原始分数，不是概率。
feature	网络从输入中提取的表示向量或特征图。
classifier	把特征映射为各类 logits 的最后一层。
CE	交叉熵，推动正确类概率增大。
KD	知识蒸馏，让学生匹配教师输出或特征。
temperature	蒸馏中平滑概率分布的温度 T 。
replay	训练新阶段时混入旧类样本。
exemplar	记忆库中代表某个旧类的样本。
catastrophic forgetting	学习新知识后旧任务性能显著下降。
CIL	测试时无任务 ID、在所有已学类别中单头分类。
AA	某阶段结束后的已学类别平均准确率。
AIA	多个增量阶段 AA 的平均。
forgetting	历史最好性能到当前性能的下降。
seed	控制初始化、打乱等伪随机过程的整数。
checkpoint	保存模型及训练状态的文件。
ablation	通过移除或替换组件判断其贡献的实验。
sanity baseline	用于验证流程与现象的基本方法，不承担强结论。
logit gap	平均新类 logits 减平均旧类 logits。
stability	保持旧知识的能力。
plasticity	学习新知识的能力。

附录 B 公式速查

线性分类器	$z = W x + b$	W 每行对应一类，z 为 logits
Softmax	$p_c = \exp(z_c) / \sum_j \exp(z_j)$	把 logits 变成概率
交叉熵	$CE = -\log p_y$	正确类概率越低，损失越大
CE 梯度	$\partial CE / \partial z_j = p_j - 1[j=y]$	抬正确类、压其他类
KD	$KL(p_{teacher}^T p_{student}^T)$	匹配教师软分布
共同平移	$\text{softmax}(z+a)=\text{softmax}(z)$	旧类内部 KD 看不到整体下移
logit gap	$\text{mean}(z_{new})-\text{mean}(z_{old})$	衡量新旧决策尺度偏移
AIA	$(1/(T+1)) \sum_t AA_t$	所有阶段准确率平均
遗忘	$\max_{\{l < t\}} a_{\{l,k\}} - a_{\{t,k\}}$	任务 k 历史最好减当前

推导重点：你最需要会的是 CE 梯度方向和 softmax 共同平移不变性。它们共同解释了“新类 CE 可以压低全部旧类，而旧类内部 KD 仍几乎不变”。

附录 C 项目文件地图与运行顺序

位置	用途
cil_restart/README.md	项目入口、状态与常用命令
增量学习交接文档.docx	旧项目事实和问题线索
增量学习代码/VGG16_CIL	旧实现，只读故障案例
cil_restart/configs/official	标准 B50-5S 方法配置
cil_restart/third_party/CLearning	固定提交的官方源码
cil_restart/results/raw	不可手改的运行日志
cil_restart/results/tidy	用于统计的整洁 CSV
cil_restart/results/tables	三线表输出
cil_restart/results/figures	PNG/PDF/SVG 图
cil_restart/results/summary	自动结论摘要
cil_restart/tests	指标、schema、损失等自动测试

推荐执行顺序

- 先运行测试，确认指标与日志处理没有被改坏。
- 运行 D0-D4 诊断或读取已有 raw，生成表 1和图 1-3。
- 解决 continuum 依赖并跑 FineTune/LwF/Replay 单种子 sanity。
- 跑 PODNet、MTD-PODNet 单种子，若 AIA 与论文偏差超过 2 个百分点，先排查。
- 复现通过后，再跑四个主方法的三种子并生成表 2-5、图 4-8。
- 最后进行 raw/tidy/table/figure 反查和组会演练。

附录 D 组会答辩题与回答骨架

问：为什么旧实验准确率突然归零？

答：仅新类 CE 抬新压旧，旧类内部 KD 对共同平移不敏感；分类器继承/更新问题进一步放大，gap 越界后旧样本被最新类吸附。

问：为什么只看最终准确率不够？

答：无法区分早期与后期、旧类与新类，也看不到遗忘过程和随机波动。

问：为什么三种子？

答：初始化、数据顺序和增强会产生波动；均值与标准差用于判断提升是否稳定。

问：D3/D4 证明了什么？

答：replay 显著缓解极端替换；短诊断尚不能证明 D4 稳定优于 D3。

问：为什么历史 checkpoint 不是 MTD？

答：类别覆盖不一致，且没有论文定义的受控教师多样性；旧实现还存在损失梯度问题。

问：MTD 的提升是否值得？

答：必须同时比较 AIA、标准差、参数、显存和训练时间；若提升小于波动而成本增加，则不值得。

问：为什么不用测试集选 checkpoint？

答：测试结果参与选择会泄漏，造成过于乐观且不可公平比较。

问：正式结果为什么还没填？

答：官方依赖 continuum==1.2.4 尚未在当前 Python 3.12 环境安装成功；在成功训练和核验前不补造数字。

附录 E 完成度清单

- 01. 我能画出图像 -> 特征 -> 分类器 -> logits -> softmax 的流程。
- 02. 我能解释 CE 梯度为什么在新类 batch 中压低旧类。
- 03. 我能推导旧类 softmax 对共同 logit 平移不变。
- 04. 我能区分 Task-IL、Domain-IL 与单头 Class-IL。
- 05. 我能解释 B50-5S 每阶段类别数和每类 20 exemplar。
- 06. 我能区分 LwF、Replay、SS-IL、PODNet 与 MTD。
- 07. 我能说明旧历史 checkpoint 为什么不是真正 MTD。
- 08. 我能用 D0-D4 的真实数字讲清类别替换机制。
- 09. 我能计算 AA、AIA、forgetting，并读懂 $\text{mean} \pm \text{std}$ 。
- 10. 我能从 raw JSON 追踪到 tidy、表格和图片。
- 11. 我能判断三线表是否混用了协议或手抄数字。
- 12. 我能从曲线、散点和混淆矩阵分别回答不同问题。
- 13. 我能说明哪些正式实验尚未完成以及原因。
- 14. 我不会把官方 MTD-SSIL 配置包装成创新。
- 15. 我能根据预设停止规则决定下一阶段研究方向。

建议：先不追求一次全会。每学完一个部分，合上讲义口头解释 3 分钟，再回到项目中找到对应文件或日志。能讲、能找、能算、能复跑，才算掌握。

附录 F 资料来源与证据边界

资料	用途与边界
增量学习交接文档.docx	旧实验时间线、现象和代码线索；其中解释需重新验证。
00_Learning_without_Forgetting.pdf	LwF 方法与无旧数据蒸馏背景。
01_Class_Incremental_Learning_with_Multi-Teacher_Distillation.pdf	MTD 方法、教师构造和论文实验。
02_Continual_Learning_With_Knowledge_Distillation_A_Survey.pdf	持续学习蒸馏方法分类与研究脉络。
HaitaoWen/CLearning	官方实现；本地固定 commit ce0789a40...48f0。
cil_restart/results/raw	D0-D4 真实诊断数字的事实源。

本讲义对论文内容采用概括和教学性转述，不替代原论文。涉及论文精确实验设置、公式符号和结论时，应回到原文和固定版本官方代码核对。

当前项目状态：D0-D4 诊断和自动表图流程已完成；标准 B50-5S 正式多方法三种子结果尚待依赖环境打通后运行。任何未运行的主结果均不得填入数值。